

Logic Programming with Focusing Proofs in Linear Logic

JEAN-MARC ANDREOLI, *E.C.R.C. GmbH Arabellastr. 17
D-8000 Munich*

Abstract

The deep symmetry of linear logic [18] makes it suitable for providing abstract models of computation, free from implementation details which are, by nature, oriented and non-symmetrical. I propose here one such model, in the area of logic programming, where the basic computational principle is

Computation = Proof search

Proofs considered here are those of the Gentzen style sequent calculus for linear logic. However, proofs in this system may be redundant, in that two proofs can be syntactically different although identical up to some irrelevant reordering or simplification of the applications of the inference rules. This leads to an untractable proof search where the search procedure is forced to make costly choices which turn out to be irrelevant. To overcome this problem, a subclass of proofs, called the 'focusing' proofs, which is both complete (any derivable formula in linear logic has a focusing proof) and tractable (many irrelevant choices in the search are eliminated when aimed at focusing proofs) is identified. The main constraint underlying the specification of focusing proofs has been to preserve the symmetry of linear logic, which is its most salient feature. In particular, dual connectives have dual properties with respect to focusing proofs.

Then, a programming language, called LinLog, consisting of a fragment of linear logic, in which focusing proofs have a more compact form, is presented. Linlog deals with formulae which have a syntax similar to that of the definite clauses and goals of Horn logic, but the crucial difference here is that it allows clauses with multiple atoms in the head, connected by the 'par' (multiplicative disjunction). It is then shown that the syntactic restriction induced by LinLog is not performed at the cost of any expressive power: a mapping from full linear logic to LinLog, preserving focusing proofs, and analogous to the normalization to clausal form for classical logic, is presented.

Keywords: Logic programming; sequent systems; proof normalization; linear logic; parallelism; concurrency.

1. Introduction

Linear logic [18] has been used in various areas of computing. Its computational appeal basically stems from its deep symmetry, expressed by the duality operator $\perp$ and the De Morgan laws (which hold without precluding the logic from being constructive). In the case of functional programming, for instance, this symmetry leads to a formulation of the cut elimination procedure (based on proof nets [22], or proof expressions [1]) in

which the role of input and output of a computation are blurred. Instead, a computation is viewed as a manipulation of resources: ‘input’ corresponds to a consumption of a resource and ‘output’ to a production, and these two operations are strictly dual. This mechanism of consumption–production of resources, which is inherently concurrent, is analogous to applications of rewrite rules, formalized in the (non-equational) framework of rewrite logic [25]. Concurrent resource manipulations also lie at the core of the applications of linear logic to Petri nets [12, 23] and can be given a metaphorical interpretation in terms of chemical reactions [9, 10].

A similar, resource based, approach to computation applies in the area of logic programming, especially to concurrent logic programming systems. In this framework, a computation consists of a set of processes running in parallel, possibly exchanging information. Each process is entirely characterized, at any given time, by its state, which is a collection of available resources. State transitions, i.e. the elementary steps in a process evolution, consist of resource manipulations, such as production, consumption, duplication. . . .

Sequent style proofs offer a satisfactory tool to formalize the history of an execution during a certain time interval. Indeed, the evolution of each computational process present at the beginning of the interval can be represented as a proof-tree, read bottom-up, whose root (conclusion) represents the state of the process at the beginning of the interval, and whose leaves (hypotheses) represent the resulting states at the end of the interval. The nodes inside the proof represent the intermediate states of the evolution of the process during the interval. There might be more than one resulting state in a transition, or none at all. This corresponds to the possibility for a process either to create several new processes or to terminate. A sequent system, which describes the correct inferences in proofs, can therefore be viewed as a formal specification of allowed process state transitions.

This process view of logic programming has already been taken in such systems as Concurrent Prolog [30, 29] Parlog [13, 15] or GHC [31]. However, these languages, based on classical logic (more precisely, its fragment restricted to Horn clauses), offer a very limited structure for representing process states. Indeed, a state must be encoded as a simple atom, i.e. a first-order term. It has been shown in [3, 4, 6, 7] that this kind of representation is inadequate both from the point of view of knowledge sharing and communication between processes, mainly because the tree structure of terms enforces an artificial hierarchical order on their components and this, in turn, results in unwanted sequentiality in the access to these components. On the other hand, linear logic offers a far richer structure for process state representation: a state is encoded as a sequent, i.e. a multiset of formulae (being unordered, multisets are more suited to concurrent access).

By precluding the possibility of freely duplicating or deleting formulae in sequents, by application of the structural rules of contraction and weakening, linear logic bestows on formulae the status of *restricted* resources. Hence, this status is explicit in linear logic, and does not derive, as in Prolog, from an interpretation of a fragment of the logic (Horn clauses and goals). Therefore, it becomes realistic to think that a proof search procedure, as ‘efficient’ in terms of resource manipulations as SLD-resolution for Horn clauses, can be devised for *full* linear logic, without any syntactical restriction. This paper attempts to devise such a procedure.

The basic procedure could be described as follows: given an initial sequent σ_0 , it incrementally builds a proof Π of σ_0 , in a Prolog-like fashion. Initially Π is assigned to a single (open) node labelled with σ_0 , and the process $\text{SEARCH}(\sigma_0)$ is started.

Procedure $\text{SEARCH}(\sigma: \text{open node of } \Pi)$

- (1) Select an instance of an inference figure of the sequent system with bottom sequent equal to σ and top sequents $\sigma_1, \dots, \sigma_n$ (with $n \geq 0$);
- (2) Expand proof Π at node σ with n branches to new open nodes labelled respectively with $\sigma_1, \dots, \sigma_n$;
- (3) For each $k = 1 \dots n$, Start $\text{SEARCH}(\sigma_k)$;

At step 3 of procedure SEARCH , the newly created open nodes $\{\sigma_k\}_{k=1 \dots n}$ can be searched simultaneously. This form of parallelism is called ‘global’, since it concerns all the branches of the proof, as opposed to ‘local’ parallelism, introduced below, and which occurs within a single branch of proof.

Step 1 of procedure SEARCH is not completely determined, in that the criterion for the selection of an inference figure is not specified. But, if we assume that all the possible choices at this step are explored (by a choice enumeration procedure for example) then all the possible proofs of the initial sequent are generated.

However, it appears that many of these generated proofs are redundant. Of course, syntactically speaking, all of them are different, since, by construction, they correspond to different sequences of application of the inference figures. But it may happen that the order in which these inferences figures are applied turns out to be irrelevant, or could be simplified, so that two (syntactically different) proofs built by the procedure SEARCH may in fact be equivalent in the following sense:

DEFINITION 1

Two proofs are said to be P-equivalent if each of them can be obtained from the other by simple permutations of inference figures and elimination or introduction of useless loops.

This equivalence relation is denoted $\Leftrightarrow$.

- Permutations of inferences in a proof are characterized by a situation of the following kind:

$$\frac{\frac{[S] \frac{\frac{\vdots}{\sigma_1} \dots \frac{\vdots}{\sigma_n}}{\sigma}}{[R] \frac{\quad}{\sigma'}}}{\sigma'} \Leftrightarrow \frac{[S] \frac{\frac{[R] \frac{\vdots}{\sigma'_1} \dots [R] \frac{\vdots}{\sigma'_n}}{\sigma'}}{\sigma'}}{\sigma'}$$

Here, the sequent σ' is derived in two different ways from the same premisses $(\sigma_i)_{i=1 \dots n}$ simply by permuting the application of inference rules R and S (this is not possible with any two inference rules R, S). From the point of view of the search process, this means that the inference rules R and S could in fact be selected (and applied) *simultaneously* at step 1 of procedure `SEARCH`, instead of sequentially, as required by the definition of this procedure. This provides a new form of parallelism, called ‘local’ parallelism, which complements the ‘global’ parallelism already mentioned above.

- Similarly, useless loops in a proof are characterized by the following situation:

$$\frac{\frac{\vdots}{\sigma} \quad \vdots}{\sigma} \Leftrightarrow \frac{\vdots}{\sigma}$$

where one sequent inside a proof is identical to the root of the proof (and hence, the sub-proof starting from that internal sequent could replace the overall proof).

Typical instances of these two cases of P-equivalence, which disturb the search procedure, are presented and analysed in the next sections.

The solution adopted here to deal with the problem of redundant P-equivalent proofs is to modify the procedure `SEARCH` in such a way that, instead of trying to build all the possible proofs of a sequent, it generates only a subset of these proofs, from which all the others could be mechanically derived. The idea is that, within this subset, the number of distinct P-equivalent proofs should be null (ideally) or at least reduced. In other words, proofs from this subset can be viewed as ‘normal’ representatives of the classes of P-equivalent proofs, and only these normal proofs are searched. I propose here a complete subset of proofs for linear logic, called the ‘focusing’ proofs, which does not rely on any syntactic restriction (unlike, for instance, resolution, which applies only to clauses in classical logic).

2. Focusing proofs

The class of focusing proof is defined below using an indirect method. It starts with the standard sequent system of linear logic, as can be found in [18] (see Fig. 1), and called, here, the ‘monadic’ system Σ_1 (this terminology is justified below). A first step of proof normalization is defined using another sequent system, called the ‘dyadic’ system Σ_2 , together with a (possibly non-deterministic) transformation from monadic to dyadic proofs such that:

If two monadic proofs can be mapped into the same dyadic proof, then they are P-equivalent.

Hence, each dyadic proof Π represents a subset of a P-equivalence class of monadic proofs, consisting of all the monadic proofs which can be mapped into Π . It constitutes a first approximation of this P-equivalence class. A better approximation is obtained during the second step of proof normalization, which is specified in exactly the same way, i.e. using another inference system, called the ‘triadic’ system Σ_3 , and a mapping from dyadic to triadic proofs verifying: if two dyadic proofs can be mapped into the same triadic proof, then they are P-equivalent.

The first normalization step is concerned with the inference rules of contraction and weakening, whereas the second normalization step deals

F, G stand for formulae, Γ, Δ stand for multisets of formulae.

- Identity $[I]$ and Cut $[\bowtie]$

$$[I] \frac{}{\vdash F, F^\perp} \quad [\bowtie] \frac{\vdash \Gamma, F \quad \vdash \Delta, F^\perp}{\vdash \Gamma, \Delta}$$

- Weakening $[<]$ and Contraction $[>]$

$$[<] \frac{\vdash \Gamma}{\vdash \Gamma, ?F} \quad [>] \frac{\vdash \Gamma, ?F, ?F}{\vdash \Gamma, ?F}$$

- Logical inference rules

$$\begin{array}{lll} [\perp] \frac{\vdash \Gamma}{\vdash \Gamma, \perp} & [\wp] \frac{\vdash \Gamma, F, G}{\vdash \Gamma, F \wp G} & [?] \frac{\vdash \Gamma, F}{\vdash \Gamma, ?F} \\ [1] \frac{}{\vdash 1} & [\otimes] \frac{\vdash \Gamma, F \quad \vdash \Delta, G}{\vdash \Gamma, \Delta, F \otimes G} & [!] \frac{\vdash ?\Gamma, F}{\vdash ?\Gamma, !F} \\ [\top] \frac{}{\vdash \Gamma, \top} & [\&] \frac{\vdash \Gamma, F \quad \vdash \Gamma, G}{\vdash \Gamma, F \& G} & [\forall] \frac{\vdash \Gamma, F[c/x]}{\vdash \Gamma, \forall x F} \\ [\oplus_l] \frac{\vdash \Gamma, F}{\vdash \Gamma, F \oplus G} & [\oplus_r] \frac{\vdash \Gamma, G}{\vdash \Gamma, F \oplus G} & [\exists] \frac{\vdash \Gamma, F[t/x]}{\vdash \Gamma, \exists x F} \end{array}$$

FIG. 1. The monadic sequent system Σ_1 .

with all the other inference rules. It could have been possible to merge the two steps and go directly from the monadic to the triadic sequent system, but at the price of some clarity.

The procedure `SEARCH` becomes much more tractable when applied to the triadic sequent system, since it avoids the generation of all the redundant P-equivalent monadic proofs which correspond to the same triadic proof.

2.1. The dyadic system Σ_2

In the monadic sequent system Σ_1 , a sequent consists of a single multiset (i.e. unordered list, denoted with Greek uppercase letters Γ, Δ) of formulae (i.e. resources). Linear logic is characterized by the fact that the two inference figures of contraction and weakening cannot be applied freely to any formula in a sequent. This means that formulae are viewed as *restricted* (bounded) resources. However, some resources may need to be *unrestricted*, so that they can be used in a proof an unbounded number of times (including 0). This is achieved by explicitly prefixing these formulae with the modality $?$, and the inference figures of contraction $[>]$ and weakening $[<]$ apply only to such modalized formulae, allowing unbounded duplication or deletion of the corresponding resource.

These two rules are immediate sources of P-equivalence. Indeed, any proof of a sequent $\Gamma, ?F$ (i.e. with at least one modalized formula), is P-equivalent to the proof simply obtained as follows:

$$[>] \frac{[<] \frac{\frac{\vdots}{\vdash \Gamma, ?F}}{\vdash \Gamma, ?F, ?F}}{\vdash \Gamma, ?F} \Leftrightarrow \frac{\vdots}{\vdash \Gamma, ?F}$$

This is a typical case of a useless loop: the two steps of contraction and weakening above just cancel each other. Similarly, contraction and weakening lead to permutations of inference figures, for instance:

$$[\wp] \frac{[<] \frac{\frac{\vdots}{\vdash \Gamma, G, H}}{\vdash \Gamma, ?F, G, H}}{\vdash \Gamma, ?F, G \wp H} \Leftrightarrow [<] \frac{[\wp] \frac{\frac{\vdots}{\vdash \Gamma, G, H}}{\vdash \Gamma, G \wp H}}{\vdash \Gamma, ?F, G \wp H}$$

The proof normalization proposed here to deal with these problems can be summarized informally as follows:

Applications of contraction and weakening should be delayed as much as possible in the search procedure and applied only when needed

Given that the search is performed from the root of the proof towards its leaves, this means that, in normal proofs, occurrences of contraction and weakening should be permuted as much as possible towards the leaves. This is always possible with weakening; contraction can also be permuted, except when it appears immediately before an occurrence of the inference figure $[\otimes]$, $[\multimap]$ or $[?]$. (Proofs are built bottom-up from the root to the leaves; an inference is ‘before’ another if it is closer to the root on the same branch.) In terms of resource manipulation, this means that an unrestricted resource should not be touched until it is actually required in the proof, or when it can be discarded because a terminal node has been reached and it was never used.

These notions are formalized in the dyadic system.

DEFINITION 2

A dyadic sequent is a pair of multisets of formulae.

The dyadic sequent made up of the pair of multisets Θ and Γ is written $\Theta : \Gamma$. In fact, it stands for the monadic sequent $? \Theta, \Gamma$ obtained by prefixing all the formulae of the first field Θ with the modality $?$. In other words, Θ represents a tank of unrestricted resources in which the proof search process can help itself at any time.

By definition, contraction and weakening could be applied freely on the formulae of Θ , but in normal proofs, they are allowed only at the leaves of a proof (for weakening) and immediately before the inference figure with which they do not permute (for contraction). This is captured in the dyadic sequent system Σ_2 , given in Fig. 2, which uses dyadic sequents. The fundamental relation between the monadic and the dyadic systems is captured by the following theorem, proved in Appendix A.1.

THEOREM 1

Let Θ and Γ be multisets of formulae. The sequent $\Theta : \Gamma$ is derivable in Σ_2 if and only if the (corresponding) sequent $? \Theta, \Gamma$ is derivable in Σ_1 . Formally,

$$\vdash \Theta : \Gamma \text{ if and only if } \vdash ? \Theta, \Gamma$$

The demonstration of this theorem is given in a constructive way, so that it would be possible (if not easy) to extract from it the specification of a mechanical (possibly non-deterministic) transformation, mapping monadic into dyadic proofs. It could then be shown that this mapping satisfies the property: if two monadic proofs can be mapped into the same dyadic proof, they are P-equivalent (simply because the demonstration of the theorem relies on trivial permutations and simplifications of inferences).

A reverse transformation from dyadic to monadic proofs is defined in Fig. 3 using simple rewrite rules on proofs. It shows that, in fact, the inference figures of the dyadic system are simply obtained from those of the monadic

F, G stand for formulae, Θ, Γ, Δ stand for multisets of formulae.

- Identity $[I]$ and Cut $[\multimap]$

$$[I] \frac{}{\vdash \Theta : F, F^\perp} \quad [\multimap] \frac{\vdash \Theta : \Gamma, F \quad \vdash \Theta : \Delta, F^\perp}{\vdash \Theta : \Gamma, \Delta}$$

- Absorption

$$[\triangleright] \frac{\vdash \Theta, F : \Gamma, F}{\vdash \Theta, F : \Gamma}$$

- Logical inference rules

$$\begin{array}{lll} [\perp] \frac{\vdash \Theta : \Gamma}{\vdash \Theta : \Gamma, \perp} & [\wp] \frac{\vdash \Theta : \Gamma, F, G}{\vdash \Theta : \Gamma, F \wp G} & [?] \frac{\vdash \Theta, F : \Gamma}{\vdash \Theta : \Gamma, ?F} \\ [1] \frac{}{\vdash \Theta : 1} & [\otimes] \frac{\vdash \Theta : \Gamma, F \quad \vdash \Theta : \Delta, G}{\vdash \Theta : \Gamma, \Delta, F \otimes G} & [!] \frac{\vdash \Theta : F}{\vdash \Theta : !F} \\ [\top] \frac{}{\vdash \Theta : \Gamma, \top} & [\&] \frac{\vdash \Theta : \Gamma, F \quad \vdash \Theta : \Gamma, G}{\vdash \Theta : \Gamma, F \& G} & [\forall] \frac{\vdash \Theta : \Gamma, F[c/x]}{\vdash \Theta : \Gamma, \forall x F} \\ [\oplus_l] \frac{\vdash \Theta : \Gamma, F}{\vdash \Theta : \Gamma, F \oplus G} & [\oplus_r] \frac{\vdash \Theta : \Gamma, G}{\vdash \Theta : \Gamma, F \oplus G} & [\exists] \frac{\vdash \Theta : \Gamma, F[t/x]}{\vdash \Theta : \Gamma, \exists x F} \end{array}$$

FIG. 2. The dyadic sequent system Σ_2 .

system by adding an extra field to each sequent, except for those dealing with the modalities.

- As expected, contraction and weakening have disappeared in the dyadic system (at least as explicit rules). Weakening is implicit in the terminal inference rules $[1]$, $[\top]$, $[I]$ (with no premiss) of the dyadic system, which have been obtained as combination of the corresponding rule of the monadic system and occurrences of weakening. Contraction is implicit in the inference rules $[\otimes]$, $[\triangleright]$, $[\multimap]$ of the dyadic system which correspond to those inferences of the monadic system with which contraction does not permute.
- Thus, the rule $[?]$ of the monadic system (also known as dereliction), which allows to effectively use a modalized formula (unrestricted resource) by stripping off its modality, is implicitly combined with a contraction in the structural rule of absorption $[\triangleright]$ of the dyadic system.

$$[\triangleright] \frac{\vdash \Theta, F : \Gamma, F}{\vdash \Theta, F : \Gamma}$$

This just means that when an unrestricted resource is to be used, it can systematically be duplicated beforehand, using contraction, even when

$$\begin{array}{l}
 [I] \frac{}{\vdash \Theta : F, F^\perp} \mapsto [<^*] \frac{[I] \frac{}{\vdash F, F^\perp}}{\vdash ?\Theta, F, F^\perp} \\
 [\bowtie] \frac{\vdash \Theta : \Gamma, F \quad \vdash \Theta : \Delta, F^\perp}{\vdash \Theta : \Gamma, \Delta} \mapsto [>^*] \frac{[\bowtie] \frac{\vdash ?\Theta, \Gamma, F \quad \vdash ?\Theta, \Delta, F^\perp}{\vdash ?\Theta, ?\Theta, \Gamma, \Delta}}{\vdash ?\Theta, \Gamma, \Delta} \\
 [\triangleright] \frac{\vdash \Theta, F : \Gamma, F}{\vdash \Theta, F : \Gamma} \mapsto [>] \frac{[?] \frac{\vdash ?\Theta, ?F, \Gamma, F}{\vdash ?\Theta, ?F, ?F, \Gamma}}{\vdash ?\Theta, ?F, \Gamma} \\
 [\perp] \frac{\vdash \Theta : \Gamma}{\vdash \Theta : \Gamma, \perp} \mapsto [\perp] \frac{\vdash ?\Theta, \Gamma}{\vdash ?\Theta, \Gamma, \perp} \\
 [\wp] \frac{\vdash \Theta : \Gamma, F, G}{\vdash \Theta : \Gamma, F \wp G} \mapsto [\wp] \frac{\vdash ?\Theta, \Gamma, F, G}{\vdash ?\Theta, \Gamma, F \wp G} \\
 [?] \frac{\vdash \Theta, F : \Gamma}{\vdash \Theta : \Gamma, ?F} \mapsto \vdash ?\Theta, ?F, \Gamma \\
 [1] \frac{}{\vdash \Theta : 1} \mapsto [<^*] \frac{[1] \frac{}{\vdash 1}}{\vdash ?\Theta, 1} \\
 [\otimes] \frac{\vdash \Theta : \Gamma, F \quad \vdash \Theta : \Delta, G}{\vdash \Theta : \Gamma, \Delta, F \otimes G} \mapsto [>^*] \frac{[\otimes] \frac{\vdash ?\Theta, \Gamma, F \quad \vdash ?\Theta, \Delta, G}{\vdash ?\Theta, ?\Theta, \Gamma, \Delta, F \otimes G}}{\vdash ?\Theta, \Gamma, \Delta, F \otimes G} \\
 [!] \frac{\vdash \Theta : F}{\vdash \Theta : !F} \mapsto [!] \frac{\vdash ?\Theta, F}{\vdash ?\Theta, !F}
 \end{array}$$

For all the other inference figures ($[\top]$, $[\&]$, $[\vee]$, $[\otimes]_l$, $[\oplus]_r$, $[\exists]$), the mapping simply replaces each dyadic sequent $\Theta : \Gamma$ in the inference figure by its corresponding monadic sequent $?\Theta, \Gamma$.

FIG. 3. Canonical injection $\Sigma_2 \mapsto \Sigma_1$.

this is not strictly needed (if the resource is not to be reused): anyway, the possibly useless contraction thus introduced will automatically be cancelled by a weakening when a leaf is reached.

- On the other hand, the rule $[?]$ of the dyadic system

$$[?] \frac{\vdash \Theta, F : \Gamma}{\vdash \Theta : \Gamma, ?F}$$

corresponds to no rule in the monadic system: mapped to monadic sequents, the top and bottom sequents of this inference figure are exactly the same. It allows to dynamically ‘fill the tank’ of unrestricted resources, but locally only, i.e. in the branch of proof where it is applied.

2.2. The triadic system Σ_3

The dyadic system deals with P-equivalent proofs of the monadic system caused by unrestricted use of the inference figures related to the modalities. But all the other inference figures are also sources of redundant P-equivalent proofs.

2.2.1. Cut and identity

One typical case of useless loops in proofs (and hence inefficiency in the search) comes from the cut rule $[\bowtie]$. Indeed, consider the following proofs:

$$[\bowtie] \frac{\frac{\frac{\vdots}{\vdash \Gamma, F \wp G} [\otimes] \frac{\frac{[I] \overline{\vdash F, F^\perp} [I] \overline{\vdash G, G^\perp}}{\vdash F^\perp \otimes G^\perp, F, G}}{\vdash \Gamma, F, G}}{\vdash \Gamma, F \wp G} \Leftrightarrow \frac{\vdots}{\vdash \Gamma, F \wp G}$$

The application of the logical inference rule $[\wp]$ at the root is immediately cancelled by the cut $[\bowtie]$ which returns the root sequent on its left premiss (hence a loop); the same problem occurs with the inference rule $[\&]$. This problem can be dealt with by an already well known proof normalization result, namely the cut-elimination theorem, proved, for the monadic system, in [18]. This theorem also holds in the dyadic system.

Furthermore, we can assume without loss of generality that the identity rule $[I]$ is always applied on atomic formulae. Indeed, non-atomic occurrences of the identity can be reduced to atomic ones, by repeatedly applying simple transformations such as

$$[I] \frac{\vdash F \wp G, F^\perp \otimes G^\perp}{\vdash F \wp G, F^\perp \otimes G^\perp} \mapsto [\wp] \frac{\frac{[I] \overline{\vdash F, F^\perp} [I] \overline{\vdash G, G^\perp}}{\vdash F, G, F^\perp \otimes G^\perp} [\otimes]}{\vdash F \wp G, F^\perp \otimes G^\perp}$$

Finally, we make the following convention:

In the sequel, we consider only proofs in the dyadic system which are cut-free and contain only atomic occurrences of the identity.

2.2.2. The problem of the principal formula

The Logical inference rules lead to multiple cases of permutations of inferences, as for instance

$$[\otimes] \frac{\frac{\vdots}{\vdash \Gamma, F, H, K} [\wp] \frac{\vdots}{\vdash \Gamma, F, \underline{H \wp K}} \quad \frac{\vdots}{\vdash \Delta, G}}{\vdash \Gamma, \Delta, \underline{F \otimes G}, H \wp K} \Leftrightarrow [\wp] \frac{\frac{\vdots}{\vdash \Gamma, F, H, K} \quad \frac{\vdots}{\vdash \Delta, G}}{\vdash \Gamma, \Delta, \underline{F \otimes G}, H, K} [\otimes] \frac{\vdots}{\vdash \Gamma, \Delta, \underline{F \otimes G}, \underline{H \wp K}}$$

From the point of view of the procedure *SEARCH*, the problem could be stated as follows. When a logical inference is to be applied at a given node, two choices must be made: (i) choice of a (non-atomic) ‘principal formula’ (underlined in the proofs above) in the sequent at that node; (ii) choice of an instance of the logical inference figure associated with the topmost connective of the selected principal formula. Although all forms of ‘don’t know’ non-determinism cannot be eliminated in these choices (a search procedure is intrinsically non-deterministic), the permutation of inferences above shows that some of these choices are not significant and either need not be considered at all or could be treated deterministically (‘don’t care’ non-determinism).

The linear connectives can be partitioned into two groups which behave differently with respect to the choice of the principal formula.

- The ‘asynchronous’ connectives:
 Multiplicative: $\perp$, $\wp$, $?$
 Additive: $\top$, $\&$, $\forall$
- The ‘synchronous’ connectives:
 Multiplicative: 1 , $\otimes$, $!$
 Additive: 0 , $\oplus$, $\exists$

This terminology is not standard and will be justified below. Notice that the dual of an asynchronous connective is synchronous and vice versa. A non-atomic formula whose topmost connective is synchronous (resp. asynchronous) is called a synchronous (resp. asynchronous) formula. The difference in search behaviour between these two groups can be characterized as follows.

If the principal formula which has been selected in a sequent is asynchronous, then there is one and only one applicable instance of the corresponding inference figure, whereas if it is synchronous, one among several (or sometimes no) instances has to be selected.

Thus, if the synchronous formula $F \otimes G$ is selected as principal formula in the sequent $\Gamma, F \otimes G$, many possible instances of the corresponding inference figure $[\otimes]$ can be applied, corresponding to the different partitions of Γ along the two branches. Similarly, a principal formula of the form $F \oplus G$ requires the choice between the left ($[\oplus_l]$) and right ($[\oplus_r]$) instances of the corresponding inference figure. On the other hand, when an asynchronous formula is selected as principal formula, there is a unique applicable instance of the corresponding inference figure and its application is therefore deterministic.

We can summarize these properties as

- Asynchronous $\mapsto$ determinism
- Synchronous $\mapsto$ non-determinism

which replaces the usual Prolog characterization where determinism is accounted for by the conjunction and non-determinism by the disjunction; linear logic features a ‘non-deterministic’ conjunction $\otimes$ and a ‘deterministic’ disjunction $\wp$.

Another, related, characterization of the difference between synchronous and asynchronous connectives concerns the reversibility of the inference figures: the conclusion of the logical inference figure associated with an asynchronous connective is derivable if and only if all its premises are derivable. This property does not hold for synchronous connectives. In fact, in the monadic system, it does not even hold for the asynchronous modality $?$: dereliction is not reversible. However, it is interesting to notice that this exception disappears in the dyadic system, where the inference rule $[?]$ is reversible. This is not surprising since, in the dyadic system, rule $[?]$ simply moves a formula prefixed with the modality inside the tank of unbounded resources, thus *enabling* future dereliction on that formula (implicit in the structural rule of absorption), but it does not actually *perform* a dereliction (so that it remains reversible).

2.2.3. Triadic sequents

Potential permutations of inferences such as those mentioned in the previous section would induce limited perturbation in a proof search procedure, were it not allowed to select the principal formula anywhere in the sequent. The proof normalization proposed here precisely imposes constraints on the way such selection is performed. It can be summarized informally as follows:

- If the sequent contains some *asynchronous* formulae (at least one), then any one of them can be *immediately* and *randomly* selected as the principal formula (‘don’t care’ non-determinism). Furthermore, as the formula thus selected is by hypothesis asynchronous, the instance of inference figure to apply is completely determined. Consequently, as long as the sequent contains an asynchronous formula, the search can be made completely deterministic.
- When all the asynchronous formulae have been decomposed, then a principal formula must be selected non-deterministically. But, as soon as one formula has been selected, the search can *focus* on it, i.e. subsequently select systematically as principal formula the subformula stemming from the initial one, *and do so as long as this subformula is synchronous*.

Asynchronous formulae are decomposed immediately as soon as they appear in the sequent (hence their name ‘asynchronous’). Synchronous formulae are delayed until all the asynchronous formulae have been decomposed, and must be non-deterministically selected to be processed: in other words, synchronous connectives synchronize the selection process and the decom-

[illegible]
$$[\otimes] \frac{\frac{\vdots}{\vdash A, F} [\otimes] \frac{[\oplus] \frac{\frac{\vdots}{\vdash B, D, G} \quad \vdots}{\vdash B, \underline{D \oplus E}, G} \quad \vdash C, H}{\vdash \underline{B \otimes C}, D \oplus E, G, H}}{\vdash A \otimes (B \otimes C), D \oplus E, F, G, H} ,$$

These notions are formalized in the triadic system.

DEFINITION 3

A triadic sequent is of one of the following two forms:

- $\Theta : \Gamma \uparrow L$ where L is an *ordered* list of formulae;
- $\Theta : \Gamma \Downarrow F$ where F is a formula;

and where Θ and Γ are multisets of formulae, Γ containing no asynchronous formula.

Thus, a triadic sequent is a dyadic sequent in which either an ordered list of formulae ($\uparrow$ -case) or one formula ($\Downarrow$ -case) has been singled out in the second field. In fact, $\Theta : \Gamma \uparrow L$ stands for the dyadic sequent $\Theta : \Gamma, L$ (in which the order of L is ‘forgotten’) and $\Theta : \Gamma \Downarrow F$ stands for the dyadic sequent $\Theta : \Gamma, F$.

F, G stand for formulae, X stands for a positive atom, Γ, Δ, Θ stand for multisets of formulae (Γ, Δ containing no asynchronous formula) and L stand for an ordered list of formulae.

- Logical inference rules

$$\begin{array}{lll}
 [\perp] \frac{\vdash \Theta : \Gamma \uparrow L}{\vdash \Theta : \Gamma \uparrow L, \perp} & [\wp] \frac{\vdash \Theta : \Gamma \uparrow L, F, G}{\vdash \Theta : \Gamma \uparrow L, F \wp G} & [?] \frac{\vdash \Theta, F : \Gamma \uparrow L}{\vdash \Theta : \Gamma \uparrow L, ?F} \\
 [\top] \frac{}{\vdash \Theta : \Gamma \uparrow L, \top} & [\&] \frac{\vdash \Theta : \Gamma \uparrow L, F \quad \vdash \Theta : \Gamma \uparrow L, G}{\vdash \Theta : \Gamma \uparrow L, F \& G} & [\forall] \frac{\vdash \Theta : \Gamma \uparrow L, F[c/x]}{\vdash \Theta : \Gamma \uparrow L, \forall x F} \\
 [1] \frac{}{\vdash \Theta : \Downarrow 1} & [\otimes] \frac{\vdash \Theta : \Gamma \Downarrow F \quad \vdash \Theta : \Delta \Downarrow G}{\vdash \Theta : \Gamma, \Delta \Downarrow F \otimes G} & [!] \frac{\vdash \Theta : \uparrow F}{\vdash \Theta : \Downarrow !F} \\
 [\otimes_l] \frac{\vdash \Theta : \Gamma \Downarrow F}{\vdash \Theta : \Gamma \Downarrow F \otimes G} & [\oplus_r] \frac{\vdash \Theta : \Gamma \Downarrow G}{\vdash \Theta : \Gamma \Downarrow F \oplus G} & [\exists] \frac{\vdash \Theta : \Gamma \Downarrow F[t/x]}{\vdash \Theta : \Gamma \Downarrow \exists x F}
 \end{array}$$

- Reaction $\uparrow$: if F is not asynchronous

$$[R\uparrow] \frac{\vdash \Theta : \Gamma, F \uparrow L}{\vdash \Theta : \Gamma \uparrow L, F}$$

- Reaction $\Downarrow$: if F is neither synchronous nor a negative atom

$$[R\Downarrow] \frac{\vdash \Theta : \Gamma \uparrow F}{\vdash \Theta : \Gamma \Downarrow F}$$

- Identities

$$[I_1] \frac{}{\vdash \Theta : X \Downarrow X^\perp} \quad [I_2] \frac{}{\vdash \Theta, X : \Downarrow X^\perp}$$

- Decisions: if F is not a positive atom

$$[D_1] \frac{\vdash \Theta : \Gamma \Downarrow F}{\vdash \Theta : \Gamma, F \uparrow} \quad [D_2] \frac{\vdash \Theta, F : \Gamma \Downarrow F}{\vdash \Theta, F : \Gamma \uparrow}$$

FIG. 4. The triadic sequent system Σ_3 .

The role of the arrows is explicated in the triadic system Σ_3 , given in Fig. 4, which uses triadic sequents:

- A sequent $\Theta : \Gamma \uparrow L$ corresponds to the case where the sequent possibly contains an asynchronous formula (in L). The third field of the sequent L acts then as a stack. Each formula from L is popped and, if it is asynchronous, it is immediately decomposed, and its components are pushed back into the stack; otherwise it is simply added to the second field of the sequent Γ which, consequently, never contains any asynchronous formula. Thus, $\uparrow$ -sequents handle the layers of asynchronous connectives, which involve only ‘don’t care’ non-determinism treated deterministically.
- A sequent $\Theta : \Gamma \Downarrow F$ corresponds to the case where all the asynchronous formulae have been decomposed and a formula F has been selected as principal formula. The subformulae of F are then systematically selected as principal formulae (since they are put back in the third field of the sequent) till a non-synchronous formula is reached. Thus, $\Downarrow$ -sequents handle the layers of synchronous connectives which are processed during the critical focusing sections. Real ‘don’t know’ non-determinism occurs only in such sections.

The fundamental relation between the dyadic and the triadic systems is captured by the following theorem, proved in Appendix A.2.

THEOREM 2

Let Θ and Γ be multisets of formulae, Γ containing no asynchronous formulae, and let L be an ordered list of formulae. The sequent $\Theta : \Gamma \uparrow L$ is derivable in Σ_3 if and only if the (corresponding) sequent $\Theta : \Gamma, L$ is derivable in Σ_2 . Formally,

$$\vdash \Theta : \Gamma \uparrow L \quad \text{if and only if} \quad \vdash \Theta : \Gamma, L$$

The demonstration of this theorem is given in a constructive way, so that it is theoretically possible to extract from it the specification of a possibly non-deterministic transformation, mapping dyadic proofs into triadic proofs and satisfying the property: if two dyadic proofs can be mapped into the same triadic proof, they are P-equivalent.

A reverse transformation from triadic to dyadic proofs is defined in Fig. 5 using simple rewrite rules on proofs. It shows that, in fact, the *logical* inference figures of the triadic system are simply obtained from those of the dyadic system by splitting the sequents with an arrow ($\uparrow$ for inference figures corresponding to asynchronous connectives and $\Downarrow$ for the synchronous case). But the triadic system also contains some specific structural rules which become trivial when mapped into the dyadic system and which handle the initialization and termination of critical focusing sections.

- The ‘reaction’ rule $[R\uparrow]$ is triggered from a sequent $\Theta:\Gamma\uparrow L, F$ when the last formula F is not asynchronous. In this case, F is just added to the second field of the sequent, Γ , for future use, when all the remaining asynchronous formulae of L will have been decomposed. Had F been asynchronous, then it would have immediately been decomposed using the Logical rule corresponding to its topmost connective.
- When all the asynchronous formulae have been decomposed in an $\uparrow$ -sequent (i.e. L is empty) a principal formula must be non-deterministically selected, starting a new critical focusing section (of $\downarrow$ -sequents). This is the role of the ‘decision’ rules. The principal formula may be picked either inside the second field of the sequent, containing the non-asynchronous formulae which have been delayed by the reaction rule $[R\uparrow]$ above (decision $[D_1]$), or in the first field, i.e. the ‘reserve tank’ of unrestricted formulae (decision $[D_2]$) where it is not discarded. The search then proceeds with a critical section focusing on the selected formula.
- The ‘reaction’ rule $[R\downarrow]$ is triggered when an asynchronous formula, or a *positive* atom, is reached at the end of a critical focusing section. The arrow is just turned upside down, which means that the critical section is terminated and the asynchronous formulae must be decomposed.
- When a *negative* atom $X^\perp$ is reached at the end of a critical focusing section, the identity must be used, so that X must be found in the rest of the sequent, either as a restricted resource in the second field (identity $[I_1]$), or as an unrestricted resource in the first field (identity $[I_2]$). This considerably reduces the amount of non-determinism involved in the critical sections.

Finally, the modalities have a peculiar behaviour in the triadic system:

- When a formula prefixed with the asynchronous modality $?$ is encountered, it is immediately stored in the first field of the sequent for possible future (unbounded) use, instead of being put back in the third field of the sequent for further decomposition of asynchronous connectives, as in the standard asynchronous case.
- The modality $!$, unlike standard synchronous connectives, terminates a critical focusing section (see rule $[!]$), but it enforces that at the moment of the interruption, the second field of the sequent is empty (i.e. the sequent contains only unbounded resources).

2.3. Summary

In this section, we have introduced the two sequent systems Σ_2 , Σ_3 together with:

- (1) An injection from Σ_2 -proofs into Σ_1 -proofs (Fig. 3) which is a canonical morphism, in that it works at the inference level (the image

of a combination of two inferences is a combination of the images of these inferences).

- (2) A projection from Σ_1 -proofs into Σ_2 -proofs, which is a mechanical, non-deterministic, transformation (specifiable from the demonstration of Theorem 1) and is a reverse of the previous canonical injection. It is defined in terms of some more or less complex rearrangement of the inferences in the proofs (permutations of inferences and eliminations or introductions of useless loops), so that if two Σ_1 -proofs are mapped into the same Σ_2 -proof, then they are necessarily P-equivalent.
- (3) A similar canonical injection from Σ_3 -proofs into Σ_2 -proofs (Fig. 5).
- (4) A similar reverse projection from Σ_2 -proofs into Σ_3 -proofs (obtained from the demonstration of Theorem 2).

This situation is summarized in Fig. 6. The term ‘focusing’ proof denotes, depending on the context, either any triadic proof (in Σ_3), or the canonical image of a triadic proof in Σ_2 , or the canonical image of such a dyadic proof in Σ_1 (in other words, the canonical injections are treated as identities).

The notion of focusing proofs is characterized by the following properties

- Focusing proofs form a complete subset of proofs for linear logic, i.e. each derivable formula in this logic has a focusing proof. This is a direct consequence of Theorems 1 and 2.

$$\begin{aligned}
 [R\uparrow] \frac{\vdash \Theta : \Gamma, F \uparrow L}{\vdash \Theta : \Gamma \uparrow L, F} &\mapsto \vdash \Theta : \Gamma, L, F \\
 [R\downarrow] \frac{\vdash \Theta : \Gamma \uparrow F}{\vdash \Theta : \Gamma \downarrow F} &\mapsto \vdash \Theta : \Gamma, F \\
 [I_1] \frac{}{\vdash \Theta : X \downarrow X^\perp} &\mapsto [I] \frac{}{\vdash \Theta : X, X^\perp} \\
 [I_2] \frac{}{\vdash \Theta, X : \downarrow X^\perp} &\mapsto [\triangleright] \frac{[I] \frac{}{\vdash \Theta, X : X, X^\perp}}{\vdash \Theta, X : X^\perp} \\
 [D_1] \frac{\vdash \Theta : \Gamma \downarrow F}{\vdash \Theta : \Gamma, F \uparrow} &\mapsto \vdash \Theta, \Gamma, F \\
 [D_2] \frac{\vdash \Theta, F : \Gamma \downarrow F}{\vdash \Theta, F : \Gamma \uparrow} &\mapsto [\triangleright] \frac{\vdash \Theta, F : \Gamma, F}{\vdash \Theta, F : \Gamma}
 \end{aligned}$$

For all the other (Logical) inference figures ($[\perp]$, $[\wp]$, $[?]$, $[\top]$, $[\&]$, $[\vee]$, $[1]$, $[\otimes]$, $[!]$, $[\oplus]$, $[\oplus_r]$, $[\exists]$) the mapping simply replaces each triadic sequent (resp. $\Theta : \Gamma \uparrow L$ or $\Theta : \Gamma \downarrow F$) in the inference figure by its corresponding dyadic sequent (resp. $\Theta : \Gamma, L$ or $\Theta : \Gamma, F$).

FIG. 5. Canonical injection $\Sigma_3 \mapsto \Sigma_2$.

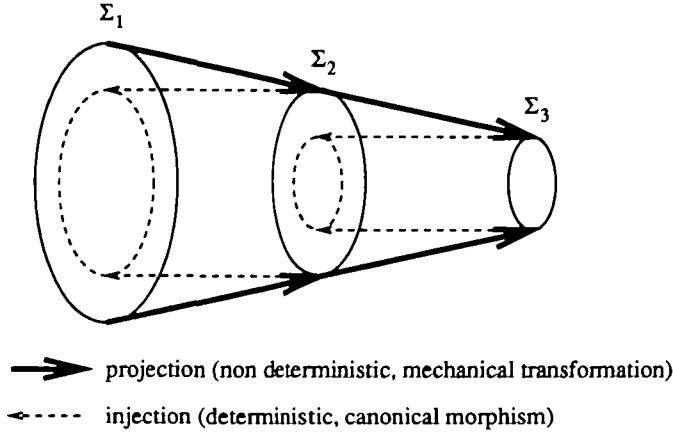


FIG. 6. The sequent systems and their relations.

- Focusing proofs respect the overall symmetry of linear logic, in that dual connectives have dual focusing properties. This duality is best visualized in the structure of the sequents of the triadic system (resp. $\Uparrow$ and $\Downarrow$), which handle the dual groups of linear connectives (resp. called here asynchronous and synchronous).

Given that many P-equivalent proofs become identical when mapped into focusing proofs, the procedure `SEARCH` defined in the introduction becomes much more tractable when constrained to aim at focusing proofs. Furthermore, given the completeness of focusing proofs, all the other proofs which could have been generated by the procedure `SEARCH` if it had not been thus constrained, could be obtained from the generated focusing proofs by application of the reverse of the projections from Σ_1 to Σ_3 .

However, notice that not all P-equivalent proofs become identical when mapped to focusing proofs. For example the following (monadic) proofs are distinct focusing proofs, although they are P-equivalent.

$$\frac{\frac{\frac{}{\vdash a^\perp, a} \quad \frac{\frac{\vdash b^\perp, b \quad \vdash c^\perp \otimes d^\perp, c, d}{\vdash c^\perp \otimes d^\perp, c, b^\perp \otimes d, b}}{\vdash c^\perp \otimes d^\perp, a^\perp \otimes c, b^\perp \otimes d, a, b}}{\vdash c^\perp \otimes d^\perp, a^\perp \otimes c, b^\perp \otimes d, a, b} \Leftrightarrow \frac{\frac{\frac{}{\vdash b^\perp, b} \quad \frac{\frac{\vdash a^\perp, a \quad \vdash c^\perp \otimes d^\perp, d, c}{\vdash c^\perp \otimes d^\perp, d, a^\perp \otimes c, a}}{\vdash c^\perp \otimes d^\perp, a^\perp \otimes c, b^\perp \otimes d, a, b}}{\vdash c^\perp \otimes d^\perp, a^\perp \otimes c, b^\perp \otimes d, a, b}$$

They just differ in the order of application of the inference rule $[\otimes]$. The cause of this problem can be identified in the triadic system: when one of the decision rules is used, to select a principal formula to focus on, this choice is completely free and is not influenced by previous steps in the proof. In fact, it may happen (as in the proofs above) that the order in which these choices are made is irrelevant and could be permuted. The triadic system is not able, in its current stage, to handle this case of permutation of inferences.

3. The logic programming language LinLog

Focusing proofs have a very simple and computationally significant interpretation, which appears clearly when they are applied to a fragment of linear logic, called LinLog, and presented below. Furthermore, the syntactic restrictions defining this fragment do not induce any restriction on the expressiveness of the language. Indeed, it is shown that any formula of linear logic can be mapped into LinLog. This mapping can be viewed as a normalization procedure, analogous to the transformation to clausal form for classical logic. The main difference is that the latter transformation preserves only provability whereas the former preserves also the structure of the (focusing) proofs.

3.1. LinLog syntax and operational semantics

3.1.1. Methods and goals

LinLog is based on two classes of formulae, called the ‘methods’ and the ‘goals’. Goals have a two-layer structure:

- Elementary goals are combinations of positive atoms connected with asynchronous connectives, and where the modality $?$ applies only to atoms. The class g of elementary goals can thus be specified by

$$g = X \mid ?X \mid \perp \mid g \wp g \mid \top \mid g \& g \mid \forall x g$$

- Goals are combinations of elementary goals connected with synchronous connectives, and where the modality $!$ applies only to elementary goals. The class G of goals can thus be specified by

$$G = g \mid !g \mid 1 \mid G \otimes G \mid 0 \mid G \oplus G \mid \exists x G$$

In other words, a goal is a positive formula (containing no negative atom) in which a synchronous connective may never occur in the scope of an asynchronous one, and no connective may occur in the scope of the modality of the same class (asynchronous or synchronous). Methods are ground formulae of the form

$$\forall \vec{x} (G \multimap X_1 \wp \dots \wp X_r)$$

where $X_1, \dots, X_r$ are positive atoms ($r \geq 1$) called the *head* of the method, and G is a goal called the *body* of the method: $\multimap$ is the linear implication defined as $A \multimap B =_{\text{def}} A^\perp \wp B$. As with Horn clauses, the universal quantifier $\forall$ in front of methods is often omitted (assuming an implicit typographical convention for variable names) and the implication is written in reverse notation:

$$X_1 \wp \dots \wp X_r \multimap G$$

DEFINITION 4

A LinLog query is a pair $\langle \mathcal{P}; g \rangle$ where $\mathcal{P}$ is a set of methods (called the program) and g is an elementary goal.

3.1.2. Triadic sequents in LinLog

Let $\langle \mathcal{P}; g \rangle$ be a LinLog query. A LinLog execution consists of searching proofs of the sequent $!\mathcal{P} \vdash g$, i.e., using one-sided sequents (monadic system), $\vdash ?\mathcal{P}^\perp, g$. Using the triadic system and Theorems 1 and 2, this can be achieved by applying procedure `SEARCH` to the sequent $\vdash \mathcal{P}^\perp : \uparrow g$. In other words, the program $\mathcal{P}$ (or, more precisely, its dual) acts as a set of unrestricted resources which provides the executing process with an everlasting source of computing energy, and the goal g acts as the initial state of the process in terms of restricted resources.

The syntax of goals and methods has been designed in such a way that triadic proofs of LinLog queries have a characteristic structure: they are repeatedly composed of:

- (1) one layer of logical inference rules mixed with the reaction rules $[R\uparrow]$ and $[R\downarrow]$, and containing only sequents of the form

$$\vdash \mathcal{P}^\perp, \Psi : \Phi \downarrow G \quad \text{or} \quad \vdash \mathcal{P}^\perp, \Psi : \Phi \uparrow \mathcal{G}$$

where Ψ and Φ are multisets of positive atoms, G is a goal and $\mathcal{G}$ an ordered list of elementary goals;

- (2) one layer of inference rules starting with a sequent of the form

$$\vdash \mathcal{P}^\perp, \Psi : \Phi \uparrow$$

where Ψ and Φ are multisets of positive atoms.

In fact, the second layer, which is the only one in which the methods of the program play a role, can be given a compact representation consisting of a single inference step, called a progression step, described below.

DEFINITION 5

Let M be the method $X_1 \wp \dots \wp X_r \multimap G$. An instance of M is a pair written $\Phi_0 \multimap G_0$ where Φ_0 is a multiset of ground positive atoms and G_0 is a ground goal, such that there exists a ground substitution σ verifying

$$\sigma \cdot \{X_1, \dots, X_r\} = \Phi_0 \quad \text{and} \quad \sigma \cdot G = G_0$$

The progression inference rule is defined by

$$[\multimap] \frac{\vdash \mathcal{P}^\perp, \Psi, \Psi_0 : \Phi \downarrow G_0}{\vdash \mathcal{P}^\perp, \Psi, \Psi_0 : \Phi, \Phi_0 \uparrow}$$

where $\Psi_0, \Phi_0 \multimap G_0$ is an instance of a method in $\mathcal{P}$.

Therefore, it is possible to specialize the inference system Σ_3 into an inference system $\Sigma_3[\mathcal{P}]$, given in Fig. 7, where the single inference rule of

Let $\mathcal{P}$ be a LinLog program, $\Psi, \Psi_0, \Phi, \Phi_1, \Phi_2$ stand for multisets of positive atoms, g, g_1, g_2 stand for elementary goals, $\mathcal{G}$ stands for an ordered list of elementary goals and G, G_1, G_2 stand for goals. X stands for a positive atom.

- Progression

$$[\circ-] \frac{\vdash \Psi, \Psi_0 : \Phi \Downarrow G_0}{\vdash \Psi, \Psi_0 : \Phi, \Phi_0 \Uparrow}$$

where $\Psi_0, \Phi_0 \circ- G_0$ is an instance of a method in $\mathcal{P}$

- Logical inference rules

$$\begin{array}{lll} [\perp] \frac{\vdash \Psi : \Phi \Uparrow \mathcal{G}}{\vdash \Psi : \Phi \Uparrow \mathcal{G}, \perp} & [\wp] \frac{\vdash \Psi : \Phi \Uparrow \mathcal{G}, g_1, g_2}{\vdash \Psi : \Phi \Uparrow \mathcal{G}, g_1 \wp g_2} & [?] \frac{\vdash \Psi, X : \Phi \Uparrow \mathcal{G}}{\vdash \Psi : \Phi \Uparrow \mathcal{G}, ?X} \\ [\top] \frac{}{\vdash \Psi : \Phi \Uparrow \mathcal{G}, \top} & [\&] \frac{\vdash \Psi : \Phi \Uparrow \mathcal{G}, g_1 \quad \vdash \Psi : \Phi \Uparrow \mathcal{G}, g_2}{\vdash \Psi : \Phi \Uparrow \mathcal{G}, g_1 \& g_2} & [\forall] \frac{\vdash \Psi : \Phi \Uparrow \mathcal{G}, g[c/x]}{\vdash \Psi : \Phi \Uparrow \mathcal{G}, \forall x g} \\ [1] \frac{}{\vdash \Psi : \Downarrow 1} & [\otimes] \frac{\vdash \Psi : \Phi_1 \Downarrow G_1 \quad \vdash \Psi : \Phi_2 \Downarrow G_2}{\vdash \Psi : \Phi_1, \Phi_2 \Downarrow G_1 \otimes G_2} & [!] \frac{\vdash \Psi : \Uparrow g}{\vdash \Psi : \Downarrow !g} \\ [\otimes_l] \frac{\vdash \Psi : \Phi \Downarrow G_1}{\vdash \Psi : \Phi \Downarrow G_1 \oplus G_2} & [\oplus_r] \frac{\vdash \Psi : \Phi \Downarrow G_2}{\vdash \Psi : \Phi \Downarrow G_1 \oplus G_2} & [\exists] \frac{\vdash \Psi : \Phi \Downarrow G[t/x]}{\vdash \Psi : \Phi \Downarrow \exists x G} \end{array}$$

- Reactions

$$[R\Uparrow] \frac{\vdash \Psi : \Phi, X \Uparrow L}{\vdash \Psi : \Phi \Uparrow L, X} \quad [R\Downarrow] \frac{\vdash \Psi : \Phi \Uparrow g}{\vdash \Psi : \Phi \Downarrow g}$$

FIG. 7. The sequent system of LinLog $\Sigma_3[\mathcal{P}]$.

progression $[\circ-]$ above replaces those of decision $[D_1], [D_2]$ and identity $[I_1], [I_2]$ of the general system. In $\Sigma_3[\mathcal{P}]$ -proofs, the occurrences of $\mathcal{P}^\perp$, which appear in the first field of all the sequents, are omitted (they are implicit).

$\Sigma_3[\mathcal{P}]$ is the inference system of the programming language LinLog, and is justified by the following theorem, which derives quite straightforwardly from Theorems 1 and 2 (see Appendix A.3 for a demonstration).

THEOREM 3

Let $\langle \mathcal{P}; g \rangle$ be a LinLog query. The sequent $!\mathcal{P} \vdash g$ is derivable in linear logic if and only if the sequent $\Uparrow g$ is derivable in $\Sigma_3[\mathcal{P}]$. Formally,

$$!\mathcal{P} \vdash g \quad \text{if and only if} \quad \vdash \mathcal{P} : \Uparrow g$$

Given a triadic sequent σ in LinLog, $\vdash \mathcal{P} \sigma$ is taken to mean that σ is derivable in $\Sigma_3[\mathcal{P}]$.

3.1.3. Computational interpretation

The inference system of LinLog has a very natural computational interpretation. It manipulates sequents which, when mapped to monadic sequents, are of the form $\vdash ?\mathcal{P}^\perp, \mathcal{C}$, where $\mathcal{C}$, called the context, is a multiset of ground goals containing at most one non-elementary goal (more precisely,

it can be shown that if $\mathcal{C}$ does contain a non-elementary goal, then the rest of the context is flat).

DEFINITION 6

A context is said to be flat if it contains only flat goals, i.e. positive atoms, possibly prefixed with the modality $?$.

The inference mechanism of LinLog can then be informally characterized in two clauses:

- If the context is not flat, then select at random a non-flat goal in the context and decompose it using the inference rule corresponding to its topmost connective. The selection step involves ‘don’t care’ non-determinism. The decomposition step may involve ‘don’t know’ non-determinism if the selected goal is not elementary (hence synchronous).
- If the context is flat, then find an instance of a method in the program such that its head matches exactly a submultiset of the atoms (modalized or not) of the context. Then replace in the context those (and only those) atoms of this submultiset which are not modalized, by the body of the method. Then proceed with the context thus updated. The selection of the method, as well as that of the submultiset of the context to match its head, involve ‘don’t know’ non-determinism.

From that point of view, standard Prolog appears as the degenerated case of LinLog in which the contexts are singletons and the heads of the methods contain themselves only one atom; in this case, the distinction between asynchronous and synchronous connectives in the goals becomes meaningless (as well as the modalities).

The computational model of LinLog is especially suited for concurrency. First, of course the different branches of proof can be searched concurrently (‘global’ parallelism). Furthermore, on one branch, several strategies can be devised for the selection of the method to apply at each step of the computation. Once the set of candidate methods (and matching subcontext) has been determined, one of them can be picked at random and applied (committed choice strategy); an other solution is to apply *simultaneously* all the candidate methods whose heads match *disjoint* submultisets of the context (‘local’ parallelism); intermediate solutions are also possible. Once the method(s) is (are) applied, the goals introduced in the context can, in turn, be decomposed in parallel. The determination of the strategy can be made at compile time, so as to make optimal use of the available parallel processing facilities.

LinLog supports various inter-process communication mechanisms. Communication is viewed here as exchange of resources between processes. Goals connected by the multiplicative connectives $\wp$ and $\otimes$ yield processes which compete for resources from their context, whereas the additive connectives $\&$ and $\oplus$ support resource sharing (in which each process gets a

separate copy of the resources). On the other hand, the asynchronous connectives $\wp$ and $\&$ support deterministic and immediate communication whereas synchronous connectives $\otimes$ and $\oplus$ lead to non-deterministic, possibly deferred, communication.

These different computational behaviours are directly significant in the framework of concurrent object-oriented systems, especially in the actors tradition. With this perspective in mind, the subset of LinLog called LO (for linear objects), in which goals are built only from the asynchronous connectives $\wp$ and $\&$, has been studied in [3, 4, 6, 7, and 8], where computational examples can be found. For instance, dynamic programming techniques find a very natural concurrent implementation in LO, as shown in [6, 8]. LO has also been applied in [7] to the optimization of Horn clause programs which have an exponential complexity when executed by the standard Prolog strategy. Using program transformation techniques, they can be converted into more tractable LO programs, switching from backward to forward chaining strategies. Various toy applications of full LinLog (including all the connectives) have been devised, but a clear characterization of the class of applications thus covered has yet to be stated.

3.2. Normalization to LinLog form

Let F be a formula. The procedure `SEARCH` could be used to build the focusing proofs of F . However, its specialized LinLog version, which has been tailored for computational efficiency, cannot be used as such if F does not satisfy the syntax of a LinLog query. It is shown here that this syntactical restriction is not bought at the price of generality. Indeed, this section presents a mechanical transformation, analogous to the transformation to clausal form for classical logic, and which maps any formula F into a LinLog query such that the focusing proofs of F are isomorphic to the LinLog proofs of the corresponding query, up to some irrelevant name conventions.

3.2.1. An example

Let $F = (a^\perp \otimes b) \wp a \wp b^\perp$. Formula F violates the syntax of LinLog elementary goals since the synchronous subformula $F_1 = a^\perp \otimes b$ and the negative atom $F_2 = b^\perp$ occur in the scope of an asynchronous connective. However, were F_1 and F_2 positive atoms, F would be an elementary goal, and the first steps of its LinLog proof would be the same as the first steps of the focusing proof of F (there is only one focusing proof here), i.e. a sequence of simple applications of the inference rules $[\wp]$ and $[R\uparrow]$:

$$\frac{\frac{\vdots}{\vdash : F_1, a, F_2 \uparrow}}{\vdots}}{\vdash : \uparrow F_1 \wp a \wp F_2}$$

320 / Focusing Proofs in Linear Logic

In the focusing proof of F , the next step consists of the selection of F_1 as principal formula, using decision rule $[D_1]$, followed by a (short) critical focusing section:

$$\frac{[D_1] \frac{[\otimes] \frac{[I_1] \overline{\vdash : a \Downarrow a^\perp} \quad \overline{\vdash : F_2 \Downarrow b}}{\vdash : a, F_2 \Downarrow F_1}}{\vdash : F_1, a, F_2 \Uparrow}}$$

We now want to map this critical section into a LinLog progression step $[\circ-]$. To achieve this, let's replace F_1 by a *new* positive atom, say u_1 , and let's try to view the formula $u_1 \circ- F_1$ as a method M_1 . Thus, the selection of formula F_1 by the decision rule $[D_1]$ above could as well be seen as the triggering of method M_1 by u_1 . Here, we simply have $M_1 = u_1 \wp a \circ- b$ and the critical section above can be mapped into the LinLog single progression step using method M_1 (as if F_2 were a positive atom):

$$[\circ-] \frac{\overline{\vdash : F_2 \Downarrow b}}{\vdash : u_1, a, F_2 \Uparrow}$$

Then, the focusing proof as well as the LinLog proof proceed in the same way:

$$[R\Downarrow] \frac{[R\Uparrow] \frac{\overline{\vdash : F_2, b \Uparrow}}{\vdash : F_2 \Uparrow b}}{\vdash : F_2 \Downarrow b}$$

At this point, the focusing proof of F proceeds in selecting formula F_2 using decision rule $[D_1]$ and starting a new critical focusing section (which completes the proof):

$$[D_1] \frac{[I_1] \overline{\vdash : b \Downarrow F_2}}{\vdash : F_2, b \Uparrow}$$

There again, we can map this critical section into a LinLog progression step $[\circ-]$: let's introduce a *new* positive atom u_2 and let's try to view $u_2 \circ- F_2$ as a method M_2 . Here $M_2 = u_2 \wp b \circ- 1$ and the critical section above is mapped into a LinLog progression step using method M_2 :

$$[\circ-] \frac{[1] \overline{\vdash : \Downarrow 1}}{\vdash : u_2, b \Uparrow}$$

Finally, let g be the formula obtained by replacing in F its subformula F_1 by u_1 and its subformula F_2 by u_2 . As intended, the formula $g = u_1 \wp a \wp u_2$ is now a LinLog elementary goal and we have mapped the focusing proof of $:\uparrow F$ (in Σ_3) into the following LinLog proof of $:\uparrow g$ (in $\Sigma_3[\mathcal{P}]$) where $\mathcal{P}$ is the LinLog program containing the methods M_1 and M_2 .

$$\frac{[\dots] \frac{[\ominus] \frac{[\dots] \frac{\vdash : \downarrow 1}{\vdash : u_2, b \uparrow}}{\vdash : u_2 \downarrow b}}{\vdash : u_1, a, u_2 \uparrow}}{\vdash : \uparrow u_1 \wp a \wp u_2}}$$

The fact that atoms u_1, u_2 are ‘new’ (i.e. do not occur anywhere else in F when they are introduced) ensures that, conversely, any LinLog proof of $:\uparrow g$ in $\Sigma_3[\mathcal{P}]$ can be mapped back into a focusing proof of $:\uparrow F$. Indeed, it ensures that the methods of $\mathcal{P}$ can only be used in the situations above, so that the proof of $:\uparrow F$ can be reconstructed. In fact, the atoms u_1, u_2 are identifiers of the syntactic occurrences of, respectively, subformulae F_1, F_2 in F , and, by definition, syntactic occurrences are unique.

The operation of ‘naming’ F_1 by u_1 (or F_2 by u_2) is strictly similar to the Skolemization step in the classical clausal transformation; only, Skolemization occurs at the term level whereas here, naming occurs at the subformula level.

3.2.2. The algorithm

We present the LinLog normalization algorithm here in the propositional case (no variables nor quantification), but it can easily be extended to the first-order case (see below). Let F_0 be a formula. The computation of its LinLog normal form, which is a LinLog query, can be informally described as follows:

- (1) Try to view F_0 as an elementary goal, i.e. determine the deepest subformulae in F_0 which preclude it from being an elementary goal in LinLog syntax.
- (2) Let g be the formula obtained by replacing in F_0 each of the subformulae computed at step 1 by a *fresh* positive atom (a different one for each subformula). By construction, g is an elementary goal, which defines the goal component of the normal form.
- (3) For each atom u introduced in F_0 to replace a subformula F , try to view (as in step 1) the formula $u \ominus F$ as a set of LinLog methods (connected by $\&$). This may lead to new replacements of subformulae by new atoms, which are recursively treated in the same way and which may produce more methods.

- (4) The set of all the methods produced at step 3 forms the program component of the normal form.

The formula $u \multimap F$ can be interpreted as the ‘definition’ of the atom u by the formula F (which u replaces in F_0). The algorithm attempts to translate this definition into a LinLog program, possibly recursively introducing new definitions. Notice that such definitions are unrestricted resources with global scope: they are just naming conventions which can be reused as many times as needed anywhere. This is why they are mapped into the program component of the query.

To specify the algorithm formally, we introduce ‘quasi-methods’, which are like methods, except that they may have an empty head, and their body is a multiset of goals (implicitly connected by $\otimes$). (In fact, elementary goals, possibly prefixed with the modality $!$, are sufficient.)

DEFINITION 7

A quasi-method is a pair $[\Phi; Y]$ where Φ is a multiset of positive atoms and Y a multiset of goals. A quasi-method is said to be strict if Φ is non-empty.

The LinLog normalization algorithm is given by the function `NORMALIZE` in Fig. 8. This algorithm uses three side functions, `NGOAL`, `NATOM` and `NMETH`, which try to view the formula passed in their argument as, respectively, an elementary goal, a positive atom and the dual of a set of quasi-methods (in the last case, the function returns this set, whose elements are taken to be implicitly connected by $\&$).

- The function `NGOAL` is trivial: it recursively scans its argument formula F till it reaches the occurrences where, for F to be an elementary goal, the subformulae at these occurrences have to be positive atoms. These subformulae are then replaced by positive atoms obtained by applying the function `NATOM` to each of them.
- The function `NATOM` is also elementary: if its argument F is already a positive atom, it simply returns it; otherwise, it introduces a new Skolem constant X , which it returns, and maps the definition of X by F , i.e. the formula $X \multimap F$, into a set of methods, which are added to $\mathcal{P}$. In fact, to avoid multiple dualization inside F , it is easier to try to map the dual formula $X^\perp \otimes F$ into the dual of a set of methods (obtaining by calling the function `NMETH`).
- The definition of the function `NMETH` must be understood as follows: any critical section of proof focusing on F can be mapped into a LinLog progression step (extended trivially to deal with non-necessarily strict quasi-methods) using one of the quasi-methods of `NMETH(F)`. Thus, for example, if F is of the form $F_1 \oplus F_2$, a critical section focusing on F necessarily goes on with a critical section focusing either on F_1 or on F_2 . Therefore, it we assume that, in both cases $i = 1, 2$, the critical section

Global $\mathcal{P}$: LinLog program;

Function NORMALIZE(F : formula) **returns** a LinLog query

$\mathcal{P} := \emptyset$;
 $g := \text{NGOAL}(F)$;
Return $\langle \mathcal{P}; g \rangle$;

Function NGOAL(F : formula) **returns** an elementary goal

Select

Case $F =$	Return
$F_1 \wp F_2$	$\text{NGOAL}(F_1) \wp \text{NGOAL}(F_2)$
$F_1 \& F_2$	$\text{NGOAL}(F_1) \& \text{NGOAL}(F_2)$
$\perp$	$\perp$
$\top$	$\top$
$?F'$	$? \text{NATOM}(F')$
Otherwise	$\text{NATOM}(F)$

Function NATOM(F : formula) **returns** a positive atom

If F is a positive atom **Then**

Return F ;

Else

Let X be a new positive atom (Skolem constant);

$\mathcal{P} := \mathcal{P} \cup \|\text{NMETH}(X^\perp \otimes F)\|$;

Return X ;

Function NMETH(F : formula) **returns** a set of quasi-methods

If F is a negative atom $X^\perp$ **Then**

Return $\{\{X; \emptyset\}\}$;

Else

Select

Case $F =$	Return
$F_1 \otimes F_2$	$\text{NMETH}(F_1) * \text{NMETH}(F_2)$
$F_1 \oplus F_2$	$\text{NMETH}(F_1) \cup \text{NMETH}(F_2)$
1	$\{\{\emptyset; \emptyset\}\}$
0	$\emptyset$
$!F'$	$\{\{\emptyset; !\text{NGOAL}(F')\}\}$
Otherwise	$\{\{\emptyset; \text{NGOAL}(F)\}\}$

FIG. 8. The LinLog normalization algorithm.

focusing on F_i can be mapped into a progression step using a quasi-method in $\text{NMETH}(F_i)$, then the critical section focusing on F can be mapped into a progression step using a quasi-method in $\text{NMETH}(F_1) \cup \text{NMETH}(F_2)$. This justifies the definition of $\text{NMETH}(F)$ in that case. The treatment of the connective $\otimes$ is a bit more complex and makes use of the operator $*$ on sets of quasi-methods, defined as follows. Let $\mathcal{S}_1, \mathcal{S}_2$ be sets of quasi-methods; $\mathcal{S}_1 * \mathcal{S}_2$ denotes the set of quasi-methods defined by pairwise combining the elements from each set:

$$\mathcal{S}_1 * \mathcal{S}_2 = \bigcup_{[\Phi_1; Y_1] \in \mathcal{S}_1 \text{ and } [\Phi_2; Y_2] \in \mathcal{S}_2} \{[\Phi_1, \Phi_2; Y_1, Y_2]\}$$

Combining two quasi-methods yields a quasi-method obtained by respectively merging their heads and their bodies.

Function NATOM may add, when called, new elements (methods) to $\mathcal{P}$ (which works as an accumulator). These methods are obtained using the operator $\| - \|$ defined as follows: let $\mathcal{S}$ be a set of *strict* quasi-methods; $\|\mathcal{S}\|$ denotes the set of corresponding methods, trivially defined as

$$\|\mathcal{S}\| = \bigcup_{[X_1, \dots, X_r; G_1, \dots, G_r] \in \mathcal{S}} \{X_1 \wp \dots \wp X_r \multimap G_1 \otimes \dots \otimes G_r\}$$

When $s=0$ in this definition, the body of the method is reduced to the logical constant 1. We have to make sure that when this operator is used in the definition of the function NATOM , its argument $\mathcal{S} = \text{NMETH}(X^\perp \otimes F)$ contains only *strict* quasi-methods (otherwise the definition above does not make sense). This directly results from a simple application of the definition of NMETH in this case:

$$\mathcal{S} = \{[X, \Phi; Y] \text{ such that } [\Phi; Y] \in \text{NMETH}(F)\}$$

Hence, the head of each quasi-method in $\mathcal{S}$ contains at least X (i.e. the defined atom) and is not empty.

The normalization algorithm is justified by the following result, proved in Appendix A.4.

THEOREM 4

Let F be a formula and $\langle \mathcal{P}; g \rangle$ be its LinLog normal form (computed by the function NORMALIZE). F is derivable if and only if g is derivable in LinLog using program $\mathcal{P}$.

$$F \text{ if and only if } \vdash_{\mathcal{P}} \uparrow g$$

The demonstration of this theorem shows that, furthermore, not only is provability preserved in the normalization, but also the structure of the focusing proofs.

The algorithm can easily be adapted to the non-propositional case. Only, when evaluating $\text{NATOM}(F)$ in the case where F is not already a positive atom, instead of simply introducing a new constant, we introduce a new functor f and return the positive atom $f(\vec{x})$ where $\vec{x}$ is the set of free variables in F . This is strictly analogous to the way Skolemization works, at the term level, in the clausal transformation. Furthermore, we have

$$\text{NGOAL}(\forall x F) = \forall x \text{NGOAL}(F)$$

$$\text{NMETH}(\exists x F) = \text{NMETH}(F)$$

3.3. Focusing and cut reduction

Focusing has been described here in a cut-free system. However, it can easily be extended to a system with cut. Indeed, consider a proof Π of a sequent $\Theta_0 : \Gamma_0$ (in the dyadic system), possibly containing cuts. Let Θ_c be the set of all the formulae of the form $H \otimes H^\perp$ such that H is a cut formula in Π . By appending Θ_c to the first field of all the sequents in Π , we obtain a proof of $\Theta_0, \Theta_c : \Gamma_0$. Now, each occurrence of a cut in this proof can be replaced as follows:

$$[\bowtie] \frac{\vdash \Theta : \Gamma, H \quad \vdash \Theta : \Delta, H^\perp}{\vdash \Theta : \Gamma, \Delta} \mapsto [\triangleright] \frac{[\otimes] \frac{\vdash \Theta : \Gamma, H \quad \vdash \Theta : \Delta, H^\perp}{\vdash \Theta : \Gamma, \Delta, H \otimes H^\perp}}{\vdash \Theta : \Gamma, \Delta}$$

The occurrence of the absorption rule $[\triangleright]$ here is justified, since the formula $H \otimes H^\perp$ which is absorbed is, by construction, in Θ_c and hence in Θ . Thus, we obtain a cut-free proof of $\Theta_0, \Theta_c : \Gamma_0$, which can in turn be focused. We can now go back to the original proof by removing Θ_c from the first field of each sequent. This is always possible except in steps where a formula in Θ_c is selected by the decision rule $[D_2]$ for decomposition. Given that the formulae of Θ_c are synchronous, and that decision rules start critical focusing sections, such steps are necessarily of the form

$$[D_2] \frac{[\otimes] \frac{\vdash \Theta : \Gamma \Downarrow H \quad \vdash \Theta : \Delta \Downarrow H^\perp}{\vdash \Theta : \Gamma, \Delta \Downarrow H \otimes H^\perp}}{\vdash \Theta : \Gamma, \Delta \Uparrow}$$

They can be replaced by introducing the following cut rule for the triadic system:

$$[\bowtie] \frac{\vdash \Theta : \Gamma \Downarrow H \quad \vdash \Theta : \Delta \Downarrow H^\perp}{\vdash \Theta : \Gamma, \Delta \Uparrow}$$

In other words, in the triadic system, the cut-rule starts a critical section focusing on the cut-formula in each of its premisses. This is one aspect of the analogy between the cut rule and the logical rule for the connective $\otimes$

mentioned in [18]. Now, from the discussion above, we can straightforwardly generalize the main result of this paper, namely that any proof (possibly containing cuts) can be mapped into a focusing proof (possibly containing focused cuts).

Naturally, the cut elimination theorem holds in the triadic system since it already holds in the dyadic system. Basically, the direct demonstration of this result, which is given in Appendix A.5, is adapted from the classical one (see [16] or [20], for example). It mainly consists of a sequence of interleaved permutation steps and reduction steps on cut formulae. In the usual case, the reduction step is possible only when the cut formula is selected as principal formula in both premisses of a cut, and the permutation steps are meant to obtain this condition. In the case of a focused proof, this condition is already ensured, but it might be violated once a reduction is performed. In fact, it can be shown that, given that the cut formula is necessarily synchronous in exactly one of the premisses (except in the trivial case where it is atomic), the reduction of its whole topmost synchronous layer can be performed ‘in one step’ in that premiss (following the critical focusing section). Matching permutations must be performed in the other premiss, but when the critical section is terminated in the first premiss, then it is the cut formula in the other premiss which becomes synchronous, and can in turn be reduced.

But there is another interesting property relating cut reduction and focusing. Indeed, the crucial part of focusing, captured by the progression rule $[\circ-]$ of the inference system of LinLog, can be expressed in terms of a cut reduction property. Consider a method $M = X_1 \wp \dots \wp X_r \circ- G$ in LinLog. A progression step using this method can be mapped into a cut using a non-logical axiom equivalent to M , namely $G^\perp, X_1, \dots, X_r$:

$$[\circ-] \frac{\frac{\vdots}{\vdash \mathcal{C}, G}}{\vdash \mathcal{C}, X_1, \dots, X_r} \mapsto [\bowtie] \frac{\vdash G^\perp, X_1, \dots, X_r, \frac{\vdots}{\vdash \mathcal{C}, G}}{\vdash \mathcal{C}, X_1, \dots, X_r}$$

Thus, we obtain a new justification of the progression rule¹ of LinLog through the following result, which is a variant of cut reduction:

Any proof using (non-logical) axioms of the form $X_1, \dots, X_r, G^\perp$ (where $X_1, \dots, X_r$ are positive atoms and G is a goal) can be transformed into a proof where each occurrence of the cut satisfies the following property: one of the premiss is a non-logical axiom and the cut formula is necessarily $G^\perp$.

This result is shown in [5] (in the framework of classical logic, but it can be straightforwardly adapted to linear logic), as a special case of a more general

¹ In fact, further refinements would be needed to account for the fact that progression can only be triggered when the context $\mathcal{C}$ is flat, and also for the fact that the head of a method may match atoms prefixed with the modality $?$, which are not removed by the progression step.

property: when cut reduction is performed in a proof containing non-logical axioms, all the cuts cannot be eliminated, but they can be reduced so as to appear in ‘stacks’ initiated by a non-logical axiom and where each cut formula in a stack comes from its initial non-logical axiom (and is not a positive atom); it can be shown that there is a direct correspondence between such stacks of cuts and critical focusing sections.

4. Conclusion and related works

It has been mentioned above that Prolog is a fragment of LinLog. Other attempts have been made to extend Prolog using sequent systems. For instance, [26] introduces the notion of ‘uniform’ proofs in intuitionistic logic (extended to the system of intuitionistic linear logic in [21]). The computational efficiency of uniform proofs basically stems from a property similar to that of LinLog proofs (in the search, a method is applied only after all the non-flat goals in the current context have been decomposed). Uniform proofs are defined for an implicational fragment of intuitionistic logic, known as Hereditary Harrop Logic,² whereas it has been shown here that the notion of LinLog proofs is a degenerated but fully representative case of the notion of focusing proofs, which apply to full linear logic. Furthermore, using the translation from intuitionistic to linear logic described in [18], uniform proofs are precisely mapped into focusing proofs.

But the ‘definite clauses’ of [26] (which correspond to the methods here) are characterized, as in Prolog, by a single atomic head; this feature simplifies considerably the mechanism of clause selection, especially in the framework of a sequential computation (although the use of lambda-terms instead of first-order terms, and the fact that clauses may be dynamically loaded in the proof, make things more complex than in the classical case). On the other hand, in a parallel environment, it has been shown in [3, 4, 6] and also in [27, 14, 11], that the formalism of multi-headed formulae (e.g. methods here) is better suited, especially for synchronization purposes. Contextual Horn clauses [27], or the logical objects of [14], or Shared Prolog [11], basically correspond to the fragment of LinLog where only the connective $\wp$ is used in goals (but of course, each of these systems also have specific features which are not accounted for in LinLog). It leads to an ‘actors’ model of computation [2, 32] where multiple independent agents perform concurrent tasks, communicating via a shared ‘blackboard’ [28] or ‘tuple space’ [17, 24]. In [6], we propose a more refined notion of blackboard

² In fact, this fragment allows all the connectives, but in such a way that it can directly be mapped into the pure implicative fragment of intuitionistic logic, without modifying substantially the shape of the uniform proofs. Thus, conjunction $\wedge$ can be mapped into stacked implications; similarly, a definite clause containing a disjunction $\vee$ in its body can be mapped into two separate clauses connected by conjunction. The mapping would not apply, were disjunction allowed to appear in the head of a clause; but this case is precisely forbidden.

called ‘forum’) which exploits not only the connective $\wp$, but also $\&$, in goals. It provides a notion of locality for the operations of consumption and production of messages in the blackboard.

Interaction nets [22] also provide a model for parallel computation. Although the theoretical justification of the system is proof reduction (cut elimination) instead of proof search, the resulting computational model is remarkably related to that of LinLog. This may be a consequence of the similarity between focusing and cut reduction mentioned in the previous section. In fact, the computational mechanism of interaction nets is, again, that of LinLog where only the connective $\wp$ is used. But interaction nets are furthermore equipped with a strong type discipline, which prevents some forms of deadlocks, and ensures strong normalization (a typical requirement in functional programming). LinLog on the other hand was designed in the perspective of possibly unsafe distributed environments, where such requirements are not realistic.

Proof nets [18] offer a desequenzialized representation of proofs. They are therefore directly relevant to the problem, addressed here, of eliminating in a search redundant P-equivalent proofs, which precisely differ only by their sequentialization. However, I could not devise a simple algorithm, such as procedure `SEARCH` given in the introduction, which could incrementally generate correct proof nets (satisfying their validity criterion: no short path), other than obtaining them from sequent proofs (but then their advantage is lost). Therefore, I preferred to stick to the sequent system approach, where sequents have a direct intuitive computational interpretation as process states, although the use of proof nets would have made the demonstration of the theorems much shorter.

The triadic system described here features sequents split into three fields in which the formulae have different behaviours, respectively, classical, linear and non-commutative linear, corresponding to various levels of restriction on the use of the structural rules of contraction, weakening and exchange. This triadic structure, which has been introduced here as an operational tool for specifying the class of focusing proofs, can also be used to provide a unified framework for various logics, as done in [19] with classical, intuitionistic and linear logics.

Acknowledgement

The material presented here is contained in a thesis of the university of Paris 6 by the author [3], which was prepared at the European Computer Industry Research Center (ECRC) in Munich. I am indebted to the members of the jury for their helpful comments and suggestions. I also thank G. Comyn and A. Herold for their encouragement and support at ECRC. Special thanks are due to R. Pareschi for the many crucial discussions which helped shape the main ideas contained in this paper.

References

- [1] S. Abramsky (1990). Computational interpretations of linear logic. Technical report, DOC, Imperial College, London, UK.
- [2] G. Agha and C. Hewitt (1987). Actors: a conceptual foundation for concurrent object-oriented programming. In B. Shriver and P. Wegner (eds) *Research Directions in Object-Oriented Programming*. MIT Press, Cambridge, MA.
- [3] J.-M. Andreoli (1990). Proposition pour une synthèse des paradigmes de la programmation logique et de la programmation par objects. Thèse d'Informatique de l'Université de Paris 6, Paris, France.
- [4] J.-M. Andreoli and R. Pareschi (1990). LO and behold! concurrent structured processes. In *Proceedings of OOPSLA/ECOOP'90*, Ottawa, Canada.
- [5] J.-M. Andreoli and R. Pareschi (1990). Logic programming with sequent systems: a linear logic approach. In *Proceedings of the Workshop on Extensions of Logic Programming*, Tübingen, Germany. Lecture Notes in Artificial Intelligence 475. Springer Verlag, Berlin.
- [6] J.-M. Andreoli and R. Pareschi (1991). Communication as fair distribution of knowledge. In *Proceedings of OOPSLA'91*, Phoenix, USA.
- [7] J.-M. Andreoli and R. Pareschi (1991). Linear Objects: Logical processes with built-in inheritance. *New Generation computing*, 9(3 + 4), 445–473.
- [8] J.-M. Andreoli and R. Pareschi (1991). Dynamic programming as multi-agent programming. In *Proceedings of the ECOOP'91 Workshop on Object-based Concurrent Computing*, Genève, Switzerland. Lecture Notes in Computer Science, Springer Verlag, Berlin.
- [9] J.-P. Banâtre, A. Coutant, and D. Le Metayer (1988). A parallel machine for multiset transformation and its programming style. *Future Generation Computer Systems*, 4(2), 133–145.
- [10] G. Berry and G. Boudol (1990). The chemical abstract machine. In *Proceedings of the 17th ACM Symposium on Principles of Programming Languages*, San Francisco, USA.
- [11] A. Brogi and P. Ciancarini (1991). The concurrent language Shared Prolog. *ACM Transactions on Programming Languages and Systems*, 13(1), 99–123.
- [12] C. Brown (1989). Relating petri nets to formulae of linear logic. Technical report, University of Edinburgh, Edinburgh, UK.
- [13] K. Clark and S. Gregory (1986). Parlog: Parallel programming in logic. *ACM Transactions on Programming Languages and Systems*, 8(1), 1–50.
- [14] J. S. Conery (1988). Logical objects. In *Proceedings of the 5th International Conference on Logic Programming*, Seattle, USA.
- [15] T. Conlon (1989). *Programming in Parlog*. Addison Wesley.
- [16] J. Gallier (1986). *Logic for Computer Science*. Harper & Row, New York.
- [17] D. Gelernter (1985). Generative communication in Linda. *ACM Transactions on Programming Languages and Systems*, 7(1), 80–113.
- [18] J.-Y. Girard (1987). Linear logic. *Theoretical Computer Science*, 50, 1–102.
- [19] J.-Y. Girard (1991). On the unity of logic. Preprint, Université de Paris 7.
- [20] J.-Y. Girard, Y. Lafont, and P. Taylor (1989). *Proofs and Types*. Cambridge University Press, Cambridge, UK.
- [21] J. S. HODAS and D. MILLER (1992). Logic programming in a fragment of intuitionistic linear logic. *Journal of Information and Computation*. To appear.
- [22] Y. Lafont (1990). Interaction nets. In *Proceedings of 17th ACM Symposium on Principles of Programming Languages*, San Francisco, USA.
- [23] N. Marti-Olliet and J. Meseguer (1989). From petri-nets to linear logic. Technical report, SRI International, Menlo Park, USA.
- [24] A. Matsuoka and S. Kawai (1988). Using tuple space communication in distributed object oriented languages. In *Proceedings of OOPSLA'88*, San Diego, USA.
- [25] J. Meseguer (1990). Rewriting as a unified model of concurrency. Technical report, SRI International, Menlo Park, USA.
- [26] D. Miller, G. Nadathur, F. Pfenning, and A. Scedrov (1991). Uniform proofs as a foundation for logic programming. *Annals of Pure and Applied Logic*, 51, 125–157.

- [27] L. Monteiro and A. Porto (1989). Contextual logic programming. In *Proceedings of the 6th International Logic Programming Conference*, Lisboa, Portugal.
- [28] H. Penny Nii (1989). Blackboard systems. In A. Barr, Cohen P., and Feigenbaum E. (eds) *The Handbook of Artificial Intelligence*. Vol. 4, pp. 1–82, Addison Wesley.
- [29] E. Shapiro (1989). The family of concurrent logic programming languages. Technical report, The Weizmann Institute of Science, Rehovot, Israel.
- [30] E. Shapiro and A. Takeuchi (1983). Object oriented programming in concurrent prolog. *New Generation Computing*, 1(1), 25–48.
- [31] K. Ueda (1986). Guarded Horn Clauses. PhD thesis, Dept of Information Engineering, University of Tokyo, Tokyo, Japan.
- [32] A. Yonezawa, E. Shibayama, Y. Honda, T. Takada, and J.-P. Briot (1990). An object-oriented concurrent computation model ABCM/1 and its description language ABCL/1. In A. Yonezawa (ed.), *ABCL, an Object-Oriented Concurrent System*. MIT Press, Cambridge, MA, pp. 13–45.

Appendix Demonstrations of the theorems

The demonstrations are generally based on inductive reasoning, using the depth of sequent proofs as the induction counter. If Π is the proof

$$\Pi = \frac{\frac{\bigwedge_{\pi_1} \sigma_1 \dots \sigma_n}{\sigma}}{\sigma}$$

then the depth of Π , written $\mu(\Pi)$, is defined inductively by

$$\mu(\Pi) = 1 + \max\{\mu(\Pi_1), \dots, \mu(\Pi_n)\}$$

The number of premisses n can be null, in that case the depth is simply 1. The induction steps of the demonstrations generally consist of a case-by-case analysis of the last inference step of a proof (the one which occurs at the root). In the sequel, we generally mention the induction hypothesis and only a few of these cases (the most significant).

Furthermore, we make the following convention: when we simply say that a sequent is derivable, or has a proof of a certain form, it is implicitly assumed to be in the sequent system (monadic, dyadic or triadic) to which the sequent belongs.

A.1. Projection $\Sigma_1 \mapsto \Sigma_2$

We have to show the result stated in Theorem 1, i.e.

$$\text{if } \vdash ?\Theta, \Gamma \text{ then } \vdash \Theta : \Gamma \quad (1)$$

(the reverse implication is immediate from Fig. 3). We make use of the following lemma.

LEMMA 5

If $\Theta \sqsubseteq \Theta'$ and $\Theta : \Gamma$ has a proof then $\Theta' : \Gamma$ has a proof of the same depth

where $\Theta \sqsubseteq \Theta'$ means that all the elements of Θ are also elements of Θ' (possibly with a different number of occurrences). This lemma results immediately from the following remark: in a proof of $\Theta : \Gamma$, the multiset Θ

appears in the first field of all the sequents. A proof of $\Theta' : \Gamma$ can therefore be obtained by simply replacing everywhere Θ by Θ' . We now come to the demonstration of the property (1).

DEMONSTRATION

Let $\mathcal{R}(n)$ be the following induction hypothesis:

If $? \Theta, \Gamma$ has a proof of depth at most n then $\Theta : \Gamma$ is derivable.

Let's assume $\mathcal{R}(n)$ and let's show $\mathcal{R}(n+1)$. Let Π be a proof of depth $n+1$ of $? \Theta, \Gamma$.

- (1) If the last step of Π is an instance of the logical inference rule $[\wp]$, then the principal formula at this step is necessarily in Γ (since the formulae in $? \Theta$ are prefixed with the modality $?$) and we have

$$\Pi = [\wp] \frac{\overset{\Pi'}{\bigwedge} \frac{\vdash ? \Theta, \Gamma', F, G}{\vdash ? \Theta, \Gamma', F \wp G}}{\vdash ? \Theta, \Gamma, F \wp G}$$

where $\Gamma = \Gamma', F \wp G$. The sub-proof Π' is of depth at most n . By the induction hypothesis $\mathcal{R}(n)$, we obtain that $\Theta : \Gamma', F, G$ is derivable, and hence, so is $\Theta : \Gamma$ by application of $[\wp]$. The inference figures $[\perp]$, $[!]$, $[\top]$, $[\wp]$, $[\vee]$, $[\oplus]$, $[\exists]$, $[I]$ are treated in the same way.

- (2) If the last step of Π is an instance of the logical inference rule $[\otimes]$ then, as above, the principal formula is necessarily in Γ and we have

$$\Pi = [\otimes] \frac{\overset{\Pi_a}{\bigwedge} \frac{\vdash ? \Theta_a, \Gamma_a, F}{\vdash ? \Theta_a, ? \Theta_b, \Gamma_a, \Gamma_b, F \otimes G} \quad \overset{\Pi_b}{\bigwedge} \frac{\vdash ? \Theta_b, \Gamma_b, G}{\vdash ? \Theta_a, ? \Theta_b, \Gamma_a, \Gamma_b, F \otimes G}}{\vdash ? \Theta_a, ? \Theta_b, \Gamma_a, \Gamma_b, F \otimes G}$$

where $\Theta = \Theta_a, \Theta_b$ and $\Gamma = \Gamma_a, \Gamma_b, F \otimes G$. By the induction hypothesis applied to Π_a (resp. Π_b), we obtain that $\Theta_a : \Gamma_a, F$ and $\Theta_b : \Gamma_b, G$ are derivable. By hypothesis, $\Theta_a \sqsubseteq \Theta$ and $\Theta_b \sqsubseteq \Theta$. By Lemma 5 we obtain that $\Theta : \Gamma_a, F$ and $\Theta : \Gamma_b, G$ are derivable, and hence, so is $\Theta : \Gamma$ by application of $[\otimes]$. The cut inference figure $[\bowtie]$ is treated in the same way.

- (3) If the last step of Π is a dereliction $[?]$, two cases must be considered

- (a) If the derelicted formula is in $? \Theta$, we have

$$\Pi = [?] \frac{\overset{\Pi'}{\bigwedge} \frac{\vdash ? \Theta', F, \Gamma}{\vdash ? \Theta', ? F, \Gamma}}{\vdash ? \Theta', F, \Gamma}$$

where $\Theta = \Theta', F$. By the induction hypothesis applied to Π' , we obtain that $\Theta':\Gamma, F$ is derivable. By Lemma 5 we obtain that $\Theta', F:\Gamma, F$ is derivable, and hence, so is $\Theta:\Gamma$ by

$$[\triangleright] \frac{\frac{\vdots}{\vdash \Theta', F:\Gamma, F}}{\vdash \Theta', F:\Gamma}$$

(b) If the derelicted formula is in Γ , we have

$$\Pi = [\?] \frac{\frac{\pi'}{\bigvee} \vdash ?\Theta, \Gamma', F}{\vdash ?\Theta, \Gamma', ?F}$$

where $\Gamma = \Gamma', ?F$. By the induction hypothesis applied to Π' , we obtain that $\Theta:\Gamma', F$ is derivable. By Lemma 5, we obtain that $\Theta, F:\Gamma', F$ is derivable, and hence, so is $\Theta:\Gamma$ by

$$[\?] \frac{[\triangleright] \frac{\frac{\vdots}{\vdash \Theta, F:\Gamma', F}}{\vdash \Theta, F:\Gamma'}}{\vdash \Theta:\Gamma', ?F}$$

The inference figures $[!]$, $[>]$, $[<]$ are treated in a similar way.

Therefore $\mathcal{R}(n+1)$ holds. By induction $\mathcal{R}(n)$ holds for all n . $\square$

A.2. Projection $\Sigma_2 \mapsto \Sigma_3$

We have to show the result stated in Theorem 2, i.e.

$$\text{if } \vdash \Theta:\Gamma, L \text{ then } \vdash \Theta:\Gamma \uparrow L \quad (2)$$

where Γ contains no asynchronous formula (the reverse implication is immediate from Fig. 5).

A.2.1. The main property

First, let's prove that property (2) is equivalent to

$$\text{if } \vdash \Theta:L \text{ then } \vdash \Theta:\uparrow L \quad (3)$$

DEMONSTRATION

- Clearly, property (3) derives from property (2) by substituting Γ with the empty multiset.

- Conversely, assume property (3) holds, and let $\Theta : \Gamma, L$ be derivable; let L' be any ordered list of the elements of Γ . By hypothesis, $\vdash \Theta : L, L'$ and, by property (3), we obtain that $\vdash \Theta : \uparrow L, L'$. Given that L' (as Γ) contains no asynchronous formula, the last steps of a proof of $\Theta : \uparrow L, L'$ can only be a sequence of applications of the reaction rule $[R\uparrow]$.

$$[R\uparrow] \frac{[R\uparrow] \frac{\vdots}{\vdash \Theta : \Gamma \uparrow L}}{\vdash \Theta : \uparrow L, L'}$$

Therefore, $\Theta : \Gamma \uparrow L$ is derivable.

Hence, property (2) and (3) are equivalent. $\square$

Now, to show that property (3) holds, we need the following ‘inversion’ lemmas, which will be shown in the next section.

LEMMA 6

Let Θ, Γ, Δ be multisets of formulae (Γ and Δ containing no asynchronous formula), let L, M be ordered lists of formulae and let F, G be formulae.

$\mathcal{L}\otimes$: If $\vdash \Theta : \Gamma \uparrow F, L$ and $\vdash \Theta : \Delta \uparrow G, M$ then $\vdash \Theta : \Gamma, \Delta, F \otimes G \uparrow L, M$

$\mathcal{L}\oplus$: If $\vdash \Theta : \Gamma \uparrow F, L$ then $\vdash \Theta : \Gamma, F \oplus G \uparrow L$

$\mathcal{L}\exists$: If $\vdash \Theta : \Gamma \uparrow F[t/x], L$ then $\vdash \Theta : \Gamma, \exists x F \uparrow L$

$\mathcal{L}\triangleright$: If $\vdash \Theta, F : \Gamma \uparrow F, L$ then $\vdash \Theta, F : \Gamma \uparrow L$

$\mathcal{L}\equiv$: If $\vdash \Theta : \Gamma \uparrow L$ and $L \equiv M$ then $\vdash \Theta : \Gamma \uparrow M$

where $L \equiv M$ means that lists L and M differ only by the order of their elements. We now come to the demonstration of the property (3).

DEMONSTRATION

Let $\mathcal{R}(n)$ be the following induction hypothesis

If $\Theta : L$ has a proof of depth at most n then $\Theta : \uparrow L$ is derivable.

Let's assume $\mathcal{R}(n)$ and let's show $\mathcal{R}(n+1)$. Let Π be a proof of depth $n+1$ of $\Theta : L$.

- (1) If the last step of Π is an instance of the logical inference rule $[\wp]$, we have

$$\Pi = [\wp] \frac{\overset{\Pi'}{\bigvee}}{\vdash \Theta : L', F \wp G} \frac{\vdash \Theta : L', F, G}{\vdash \Theta : L', F \wp G}$$

where $L \equiv L', F \wp G$. The sub-proof Π' is of depth at most n . By the induction hypothesis $\mathcal{R}(n)$, we obtain that $\Theta : \uparrow L', F, G$ is derivable, and hence, so is $\Theta : \uparrow L', F \wp G$ by application of $[\wp]$. Given that $L \equiv L', F \wp G$, by property $\mathcal{L} \equiv$ of Lemma 6, we obtain that $\Theta : \uparrow L$ is derivable. The inference figures $[\perp]$, $[?]$, $[\top]$, $[\&]$, $[\vee]$ are treated in the same way.

- (2) If the last step of Π is an instance of the logical inference rule $[\otimes]$, we have

$$\Pi = [\otimes] \frac{\frac{\overset{\Pi_a}{\bigwedge}}{\vdash \Theta : L_a, F} \quad \frac{\overset{\Pi_b}{\bigwedge}}{\vdash \Theta : L_b, G}}{\vdash \Theta : L_a, L_b, F \otimes G}$$

where $L \equiv L_a, L_b, F \otimes G$. By the induction hypothesis applied to Π_a (resp. Π_b), we obtain that $\Theta : \uparrow F, L_a$ and $\Theta : \uparrow G, L_b$ are derivable. By property $\mathcal{L} \otimes$ of Lemma 6, we obtain that $\Theta : F \otimes G \uparrow L_a, L_b$ is derivable, and hence, so is $\Theta : \uparrow L_a, L_b, F \otimes G$ by application of $[R\uparrow]$. Finally, given that $L \equiv L_a, L_b, F \otimes G$, by property $\mathcal{L} \equiv$ of Lemma 6, we obtain that $\Theta : \uparrow L$ is derivable.

The inference figures $[\oplus_l]$, $[\oplus_r]$, $[\exists]$ are treated in the same way, using properties $\mathcal{L} \oplus$ and $\mathcal{L} \exists$ of Lemma 6.

- (3) If Π is simply

$$\Pi = [1] \frac{}{\vdash \Theta : 1}$$

then $L = 1$ and $\Theta : \uparrow L$ is derivable by

$$[R\uparrow] \frac{[D_l] \frac{[1] \frac{}{\vdash \Theta : \downarrow 1}}{\vdash \Theta : 1 \uparrow}}{\vdash \Theta : \uparrow 1}}$$

- (4) If the last step of Π is an instance of the logical inference rule $[!]$, we have

$$\Pi = [!] \frac{\frac{\overset{\Pi'}{\bigwedge}}{\vdash \Theta : F}}{\vdash \Theta : !F}$$

and $L = !F$. By the induction hypothesis applied to Π' , we obtain that $\Theta : \uparrow F$ is derivable and hence, so is $\Theta : \uparrow L$ by

$$\frac{[R\uparrow] \frac{[D_1] \frac{[!] \frac{\vdots}{\vdash \Theta : \uparrow F}}{\vdash \Theta : \downarrow !F}}{\vdash \Theta : !F\uparrow}}{\vdash \Theta : \uparrow !F}}$$

(5) If the last step of Π is an instance of the absorption rule $[\triangleright]$, we have

$$\Pi = [\triangleright] \frac{\overset{\Pi'}{\bigvee} \vdash \Theta', F : L, F}{\vdash \Theta', F : L}$$

where $\Theta = \Theta', F$. By the induction hypothesis applied to Π' , we obtain that $\Theta', F : \uparrow F, L$ is derivable. By property $\mathcal{L}\triangleright$ of Lemma 6, we obtain that $\Theta', F : \uparrow L$, i.e. $\Theta : \uparrow L$, is derivable.

(6) Finally, if Π is simply

$$\Pi = [I] \frac{}{\vdash \Theta : X, X^\perp}$$

then $L = X, X^\perp$ and $\Theta : \uparrow L$ is derivable by

$$[R\uparrow^2] \frac{[D_1] \frac{[I_1] \frac{}{\vdash \Theta : X \downarrow X^\perp}}{\vdash \Theta : X, X^\perp \uparrow}}{\vdash \Theta : \uparrow X, X^\perp}}$$

Therefore $\mathcal{R}(n+1)$ holds. by induction $\mathcal{R}(n)$ holds for all n . $\square$

A.2.2. The inversion lemma $\mathcal{L}\equiv$

We show here property $\mathcal{L}\equiv$ of Lemma 6, i.e.

$$\text{if } \vdash \Theta : \Gamma \uparrow L \text{ and } L \equiv M \text{ then } \vdash \Theta : \Gamma \uparrow M$$

DEMONSTRATION

The proof is long but not difficult. It is sketched here.

- It is first shown that the logical inference rules concerning $\uparrow$ -sequents in the triadic system apply anywhere in the last field of these sequents (not

only to the last formula). In other words,

$$\left\{ \begin{array}{ll} \vdash \Theta : \Gamma \uparrow L, \perp, M & \text{if } \vdash \Theta : \Gamma \uparrow L, M \\ \vdash \Theta : \Gamma \uparrow L, F \wp G, M & \text{if } \vdash \Theta : \Gamma \uparrow L, F, G, M \\ \vdash \Theta : \Gamma \uparrow L, ?F, M & \text{if } \vdash \Theta, F : \Gamma \uparrow L, M \\ \vdash \Theta : \Gamma \uparrow L, \top, M & \\ \vdash \Theta : \Gamma \uparrow L, F \wp G, M & \text{if } \vdash \Theta : \Gamma \uparrow L, F, M \text{ and } \vdash \Theta : \Gamma \uparrow L, G, M \\ \vdash \Theta : \Gamma \uparrow L, \forall x F, M & \text{if } \vdash \Theta : \Gamma \uparrow L, F[c/x], M \\ \vdash \Theta : \Gamma, F \uparrow L, M & \text{if } \vdash \Gamma \uparrow L, F, M \text{ and } F \text{ is not asynchronous} \end{array} \right. \quad (4)$$

All these properties are shown by induction of the complexity of M (i.e. the sum of the complexities of its elements).

- Then it is shown that

$$\text{if } \vdash \Theta : \Gamma \uparrow L, F, M \text{ then } \vdash \Theta : \Gamma \uparrow L, M, F \quad (5)$$

This is also shown by induction on the complexity of M using, for each case of the last formula of M , the corresponding case in property (4).

- From property (5), it can easily be shown by induction that provability in the triadic system is preserved under any transposition in the last field of the sequent. More generally, provability is preserved under any permutation. This is precisely the content of property $\mathcal{L} \equiv$ of Lemma 6. $\square$

If $\Theta : \Gamma, F \uparrow L$ is derivable (where F is not asynchronous), then so is $\Theta : \Gamma \uparrow L, F$ by application of $[R \uparrow]$, and, by property $\mathcal{L} \equiv$, so is $\Theta : \Gamma \uparrow F, L$. But we can state a more specific property, which is used below:

LEMMA 7

Let Θ, Γ be multisets of formulae, L be an ordered list of formulae, and F be a formula (Γ, F containing no asynchronous formula). If $\Theta : \Gamma, F \uparrow L$ has a proof of depth n then $\Theta : \Gamma \uparrow F, L$ has a proof of depth at most $n + 1$.

This is shown by a simple induction on the complexity of L .

A.2.3. The other inversion lemmas

We now show property $\mathcal{L} \otimes$ of Lemma 6. The remaining properties in this lemma ($\mathcal{L} \oplus$, $\mathcal{L} \exists$ and $\mathcal{L} \triangleright$) are shown in the same way. Thus, we have to show:

$$\text{if } \vdash \Theta : \Gamma \uparrow F, L \text{ and } \vdash \Theta : \Delta \uparrow G, M \text{ then } \vdash \Theta : \Gamma, \Delta, F \otimes G \uparrow L, M$$

where Γ and Δ contain no asynchronous formula. This is shown by induction, but the problem with this induction is that it has to deal with both $\uparrow$ and $\downarrow$ -sequents in an interleaved way (whereas the proof of $\mathcal{L} \equiv$ was only concerned with $\uparrow$ -sequents).

DEMONSTRATION

Let $\mathcal{R}(n)$ be the following induction hypothesis.

$$\mathcal{R}(n) = \mathcal{R}^\uparrow(n) \quad \text{and} \quad \mathcal{R}^\downarrow(n-1)$$

where

- $\mathcal{R}^\uparrow(n)$ is the property:
If $\Theta : \Gamma \uparrow F, L$ and $\Theta : \Delta \uparrow G, M$ have proofs, whose total depth does not exceed n , then $\Theta : \Gamma, \Delta, F \otimes G \uparrow L, M$ is derivable.
- $\mathcal{R}^\downarrow(n)$ is the property:
If F is a synchronous formula or a negative atom and if $\Theta : \Gamma, F \downarrow H$ and $\Theta : \Delta \uparrow G$ have proofs, whose total depth does not exceed n , then $\Theta : \Gamma, \Delta, F \otimes G \downarrow H$ is derivable.

By ‘total’ depth of several proofs, we mean the sum of their depths. Let’s assume $\mathcal{R}(n)$, i.e. $\mathcal{R}^\uparrow(n)$ and $\mathcal{R}^\downarrow(n-1)$, and let’s show $\mathcal{R}(n+1)$, i.e. $\mathcal{R}^\uparrow(n+1)$ and $\mathcal{R}^\downarrow(n)$

- First, let’s show $\mathcal{R}^\uparrow(n+1)$. Let Π_1 and Π_2 be proofs of, respectively, $\Theta : \Gamma \uparrow F, L$ and $\Theta : \Delta \uparrow G, M$ such that $\mu(\Pi_1) + \mu(\Pi_2) = n + 1$.

- (1) If L or M is not empty: say $L = L', H \wp K$ (the other cases for the last formula of L are similar). Therefore

$$\Pi_1 = [\wp] \frac{\overline{\Pi_1} \quad \vdash \Theta : \Gamma \uparrow F, L', H, K}{\vdash \Theta : \Gamma \uparrow F, L', H \wp K}$$

By construction, $\mu(\Pi_1') + \mu(\Pi_2) \leq n$. By the induction hypothesis $\mathcal{R}^\uparrow(n)$ we obtain that $\Theta : \Gamma, \Delta, F \otimes G \uparrow L', H, K, M$ is derivable, and hence so is $\Theta : \Gamma, \Delta, F \otimes G \uparrow L, M$ by application of property $\mathcal{L} \equiv$ and the inference rule $[\wp]$.

- (2) If both L and M are empty and neither F nor G is a synchronous formula or a negative atom, then $\Theta : \Gamma, \Delta, F \otimes G \uparrow$ is derivable by

$$[D_1] \frac{[\otimes] \frac{[R\downarrow] \frac{\overline{\Pi_1} \quad \vdash \Theta : \Gamma \uparrow F}{\vdash \Theta : \Gamma \downarrow F} \quad [R\downarrow] \frac{\overline{\Pi_2} \quad \vdash \Theta : \Delta \uparrow G}{\vdash \Theta : \Delta \downarrow G}}{\vdash \Theta : \Gamma, \Delta \downarrow F \otimes G}}{\vdash \Theta : \Gamma, \Delta, F \otimes G \uparrow}}$$

The inferences $[R\downarrow]$ above are valid only because of the hypotheses on F and G .

- (3) If both L and M are empty and F is a synchronous formula or a negative atom, then the last inference step of Π_1 is an instance of the reaction rule $[R\uparrow]$, preceded by an instance of a decision rule $[D_1]$ or $[D_2]$. Three cases are possible:

(a) If

$$\Pi_1 = [R\uparrow] \frac{\overline{\bigwedge}^{\Pi_1} \frac{[D_1] \frac{\vdash \Theta : \Gamma', F \Downarrow H}{\vdash \Theta : \Gamma', H, F\uparrow}}{\vdash \Theta : \Gamma', H \uparrow F}}$$

where $\Gamma = \Gamma', H$, then $\mu(\Pi'_1) + \mu(\Pi_2) \leq n - 1$. By the induction hypothesis $\mathcal{R}^\sharp(n - 1)$ we obtain that $\Theta : \Gamma', \Delta, F \otimes G \Downarrow H$ is derivable, and hence, so is $\Theta : \Gamma, \Delta, F \otimes G \uparrow$ by application of $[D_1]$.

(b) If

$$\Pi_1 = [R\uparrow] \frac{\overline{\bigwedge}^{\Pi_1} \frac{[D_2] \frac{\vdash \Theta', H : \Gamma, F \Downarrow H}{\vdash \Theta', H : \Gamma, F\uparrow}}{\vdash \Theta', H : \Gamma \uparrow F}}$$

where $\Theta = \Theta', H$, then $\mu(\Pi'_1) + \mu(\Pi_2) \leq n - 1$. By the induction hypothesis $\mathcal{R}^\sharp(n - 1)$ we obtain that $\Theta : \Gamma, \Delta, F \otimes G \Downarrow H$ is derivable, and hence, so is $\Theta : \Gamma, \Delta, F \otimes G \uparrow$ by application of $[D_2]$.

(c) If

$$\Pi_1 = [R\uparrow] \frac{\overline{\bigwedge}^{\Pi_1} \frac{[D_1] \frac{\vdash \Theta : \Gamma \Downarrow F}{\vdash \Theta : \Gamma, F\uparrow}}{\vdash \Theta : \Gamma \uparrow F}}$$

then, by symmetry, we can assume that G is a synchronous formula or a negative atom and that

$$\Pi_2 = [R\uparrow] \frac{\overline{\bigwedge}^{\Pi_2} \frac{[D_1] \frac{\vdash \Theta : \Delta \Downarrow G}{\vdash \Theta : \Delta, G\uparrow}}{\vdash \Theta : \Delta \uparrow G}}$$

Therefore, $\Theta : \Gamma, \Delta, F \otimes G \uparrow$ is derivable by

$$[D_1] \frac{[\otimes] \frac{\frac{\pi_1}{\vdash \Theta : \Gamma \Downarrow F} \quad \frac{\pi_2}{\vdash \Theta : \Delta \Downarrow G}}{\vdash \Theta : \Gamma, \Delta \Downarrow F \otimes G}}{\vdash \Theta : \Gamma, \Delta, F \otimes G \uparrow}$$

Therefore $\mathcal{R}^\dagger(n+1)$ holds.

Now, let's show $\mathcal{R}^\dagger(n)$. Let F be a synchronous formula or a negative atom and let Π_1 and Π_2 be proofs of, respectively, $\Theta : \Gamma, F \Downarrow H$ and $\Theta : \Delta \uparrow G$ such that $\mu(\Pi_1) + \mu(\Pi_2) = n$.

- (1) If H is synchronous: say $H = H_a \oplus H_b$ (the other cases for H are similar). Therefore, for example,

$$\Pi_1 = [\oplus_l] \frac{\frac{\pi_1}{\vdash \Theta : \Gamma, F \Downarrow H_a}}{\vdash \Theta : \Gamma, F \Downarrow H_a \oplus H_b}$$

By construction, $\mu(\Pi'_1) + \mu(\Pi_2) \leq n-1$. By the induction hypothesis $\mathcal{R}^\dagger(n-1)$ we obtain that $\Theta : \Gamma, \Delta, F \otimes G \Downarrow H_a$ is derivable, and hence so is $\Theta : \Gamma, \Delta, F \otimes G \Downarrow H$ by application of $[\oplus_l]$.

- (2) If H is not synchronous, then H cannot be a negative atom $X^\perp$, since, in that case, we would have necessarily

$$\Pi_1 = [I_l] \frac{}{\vdash \Theta : X \Downarrow X^\perp}$$

This would require $\Gamma = \emptyset$ and $F = X$. So F would be a positive atom, in contradiction with the hypothesis that F is a synchronous formula or a negative atom. Therefore, H being neither synchronous nor a negative atom, we have

$$\Pi_1 = [R\Downarrow] \frac{\frac{\pi_1}{\vdash \Theta : \Gamma, F \uparrow H}}{\vdash \Theta : \Gamma, F \Downarrow H}$$

Therefore, $\Theta : \Gamma, F \uparrow H$ has a proof Π'_1 . By Lemma 7, given that F is not asynchronous, we obtain that $\Theta : \Gamma \uparrow F, H$ has a proof of depth at most $\mu(\Pi'_1) + 1$, i.e. $\mu(\Pi_1)$. Therefore, $\Theta : \Gamma \uparrow F, H$ and $\Theta : \Delta \uparrow G$ have proofs, whose total depth does not exceed $\mu(\Pi_1) + \mu(\Pi_2)$, which is equal to n , by hypothesis. By the induction hypothesis $\mathcal{R}^\dagger(n)$ we obtain that $\Theta : \Gamma, \Delta, F \otimes G \uparrow H$ is derivable, and hence so is $\Theta : \Gamma, \Delta, F \otimes G \Downarrow H$ by application of $[R\Downarrow]$.

Therefore $\mathcal{R}^\dagger(n)$ holds.

Therefore, both $\mathcal{R}^\dagger(n+1)$ and $\mathcal{R}^\downarrow(n)$ hold. Therefore $\mathcal{R}(n+1)$ holds. By induction $\mathcal{R}(n)$ holds for all n . $\square$

A.3. Soundness and completeness of LinLog

We have to show the result stated in Theorem 3, i.e.

$$!\mathcal{P} \vdash g \quad \text{iff} \quad \vdash_{\mathcal{P}} \uparrow g$$

We have

- $!\mathcal{P} \vdash g$ if and only if [transposition to monadic sequents]
- $\vdash ?\mathcal{P}^\perp, g$ if and only if [projection to dyadic sequents, Theorem 1]
- $\vdash \mathcal{P}^\perp : g$ if and only if [projection to triadic sequents, Theorem 2]
- $\vdash \mathcal{P}^\perp : \uparrow g$

Therefore, we have to show that

$$\vdash \mathcal{P}^\perp : \uparrow g \quad \text{iff} \quad \vdash_{\mathcal{P}} \uparrow g$$

This is achieved by a trivial induction on the depth of the proof. The induction hypothesis is

If $\mathcal{P}^\perp, \Psi : \Phi \uparrow \mathcal{G}$ (resp. $\mathcal{P}^\perp, \Psi : \Phi \downarrow G$) has a proof of depth at most n in Σ_3 , then $\Psi : \Phi \uparrow \mathcal{G}$ (resp. $\Psi : \Phi \downarrow G$) is derivable in $\Sigma_3[\mathcal{P}]$.

(the hypothesis for the reverse implication is similar). The only non-obvious case of the induction comes from the progression rule $[\circ-]$ and is solved by the following lemma.

LEMMA 8

Let Ψ, Φ be multisets of positive atoms.

- If $\Psi_0, \Phi_0 \circ- G_0$ is an instance of a method in $\mathcal{P}$, and $\vdash \mathcal{P}^\perp, \Psi, \Psi_0 : \Phi \downarrow G_0$, then $\vdash \mathcal{P}^\perp, \Psi, \Psi_0 : \Phi, \Phi_0 \uparrow$.
- Conversely, if $\vdash \mathcal{P}^\perp, \Psi : \Phi \uparrow$ then there exists an instance $\Psi_0, \Phi_0 \circ- G_0$ of a method in $\mathcal{P}$, and multisets of positive atoms Ψ' and Φ' such that

$$\Psi = \Psi_0, \Psi' \quad \text{and} \quad \Phi = \Phi_0, \Phi' \quad \text{and} \quad \vdash \mathcal{P}^\perp, \Psi', \Psi_0 : \Phi' \downarrow G_0$$

DEMONSTRATION

- Let M be a method in $\mathcal{P}$, say

$$M = \forall x p(x) \wp q(x) \circ- G$$

and let $\Psi_0, \Phi_0 \circ- G_0$ be an instance of M , say

$$\Psi_0 = p(t) \quad \Phi_0 = q(t) \quad G_0 = G[t/x]$$

where t is a ground term. Let's assume that $\mathcal{P}^\perp, \Psi, \Psi_0 : \Phi \downarrow G_0$ is

derivable. Then so is $\mathcal{P}^\perp, \Psi, \Psi_0 : \Phi, \Phi_0 \uparrow$ by

$$\frac{\frac{\frac{\Pi'}{\bigwedge}}{[\exists]} \frac{\vdash \mathcal{P}^\perp, \Psi, \Psi_0 : \Phi, \Phi_0 \Downarrow p(t)^\perp \otimes q(t)^\perp \otimes G_0}{\vdash \mathcal{P}^\perp, \Psi, \Psi_0 : \Phi, \Phi_0 \Downarrow M^\perp}}{[D_2]} \frac{}{\vdash \mathcal{P}^\perp, \Psi, \Psi_0 : \Phi, \Phi_0 \uparrow}$$

where Π' is the following proof:

$$[\otimes] \frac{[\otimes] \frac{[I_2] \frac{}{\vdash \mathcal{P}^\perp, \Psi, p(t) : \downarrow p(t)^\perp} [\otimes] \frac{[I_1] \frac{}{\vdash \Theta : q(t) \Downarrow q(t)^\perp} \frac{\vdash \Theta : \Phi \Downarrow G_0}{\vdash \Theta : \Phi, q(t) \Downarrow q(t)^\perp \otimes G_0}}{\vdash \mathcal{P}^\perp, \Psi, p(t) : \Phi, q(t) \Downarrow p(t)^\perp \otimes q(t)^\perp \otimes g_0}}{\vdash \mathcal{P}^\perp, \Psi, p(t) : \Phi, q(t) \Downarrow p(t)^\perp \otimes q(t)^\perp \otimes g_0}$$

where Θ stands for $\mathcal{P}^\perp, \Psi, p(t)$.

- Conversely, let's assume that $\mathcal{P}^\perp, \Psi : \Phi \uparrow$ has a proof Π . The last step of Π is necessarily an instance of a decision rule $[D_1]$ or $[D_2]$. As the selected formula in these rules must not be a positive atom and Ψ, Φ contain only positive atoms, we have necessarily

$$\Pi = [D_2] \frac{\frac{\Pi'}{\bigwedge}}{\vdash \mathcal{P}^\perp, \Psi : \Phi \Downarrow M^\perp} \frac{}{\vdash \mathcal{P}^\perp, \Psi : \Phi \uparrow}$$

where M is a method in $\mathcal{P}$, say

$$M = \forall x p(x) \wp q(x) \multimap G$$

By construction, Π' starts a critical focusing section, and is therefore necessarily of the form

$$\Pi' = [\exists] \frac{\frac{\Pi''}{\bigwedge}}{\vdash \mathcal{P}^\perp, \Psi : \Phi \Downarrow p(t)^\perp \otimes q(t)^\perp \otimes G_0} \frac{}{\vdash \mathcal{P}^\perp, \Psi : \Phi \Downarrow M^\perp}$$

where t is a ground term (selected at step $[\exists]$) and $G_0 = G[t/x]$. Let Θ be $\mathcal{P}^\perp, \Psi$. As the critical section proceeds, we have necessarily

$$\Pi'' = [\otimes] \frac{\frac{\frac{\Pi_p}{\bigwedge}}{\vdash \Theta : \Phi_p \Downarrow p(t)^\perp} [\otimes] \frac{\frac{\frac{\Pi_q}{\bigwedge}}{\vdash \Theta : \Phi_q \Downarrow q(t)^\perp} \frac{\frac{\Pi_0}{\bigwedge}}{\vdash \Theta : \Phi' \Downarrow G_0}}{\vdash \Theta : \Phi_q, \Phi' \Downarrow q(t)^\perp \otimes G_0}}{\vdash \Theta : \Phi_p, \Phi_q, \Phi' \Downarrow p(t)^\perp \otimes q(t)^\perp \otimes G_0}$$

where $\Phi = \Phi_p, \Phi_q, \Phi'$. Now, on each of the two leftmost branches, a negative atom is reached which terminates the critical section. Hence

one of the identities ($[I_1]$ or $[I_2]$) must be used, say

$$\Pi_p = [I_2] \frac{}{\vdash \Theta : \Downarrow p(t)^\perp} \quad \Pi_q = [I_1] \frac{}{\vdash \Theta : q(t) \Downarrow q(t)^\perp}$$

where $p(t) \in \Theta$. Therefore, $\Phi_p = \emptyset$ and $\Phi_q = q(t)$, and hence, $\Phi = q(t), \Phi'$. Furthermore, since $p(t)$ is in $\mathcal{P}^\perp$, Ψ and $\mathcal{P}^\perp$ cannot contain positive atoms, $p(t)$ must be in Ψ and we obtain that $\Psi = p(t), \Psi'$. Finally we have

$$\begin{aligned} \Phi &= \Phi_0, \Phi' \\ \Psi &= \Psi_0, \Psi' \end{aligned}$$

where $\Psi_0 = p(t)$ and $\Phi_0 = q(t)$. Therefore $\Psi_0, \Phi_0 \multimap G_0$ is an instance of M . Furthermore, Π_0 is a proof of $\mathcal{P}^\perp, \Psi : \Phi' \Downarrow G_0$.

A.4. Normalization to LinLog form

We have to show the result stated in Theorem 4, i.e.

$$\vdash F \text{ iff } \vdash_{\mathcal{P}} \Uparrow g$$

where $\langle \mathcal{P}; g \rangle = \text{NORMALIZE}(F)$.

Let's extend the definition of the function `NORMALIZE` to triadic sequents $\Theta : \Gamma \Uparrow L$ (where Γ contains no asynchronous formula). It returns an extended LinLog query consisting of a LinLog program and a sequent $\Psi : \Phi \Uparrow \mathcal{G}$ where Ψ and Φ are multisets of atoms and $\mathcal{G}$ is an ordered list of elementary goals.

Function `NORMALIZE`($\Theta : \Gamma \Uparrow L$) **returns** an extended LinLog query

$\mathcal{P} := \emptyset;$
 $\mathcal{G} := \text{NGOAL}(L); \Phi := \text{NATOM}(\Gamma); \Psi := \text{NATOM}(\Theta);$
Return $\langle \mathcal{P}; \Psi : \Phi \Uparrow \mathcal{G} \rangle$

Remark: in this definition, when the function `NATOM` or `NGOAL` is applied to a list (resp. multiset) of formulae, it returns the list (resp. multiset) of images of these formulae. We now have to show the more general property

$$\vdash \Theta : \Gamma \Uparrow L \text{ iff } \vdash_{\mathcal{P}} \Psi : \Phi \Uparrow \mathcal{G}$$

where $\langle \mathcal{P}; \Psi : \Phi \Uparrow \mathcal{G} \rangle = \text{NORMALIZE}(\Theta : \Gamma \Uparrow L)$. Let's show here the direct implication (the converse is similar).

DEMONSTRATION

Let $\mathcal{R}(n)$ be the induction hypothesis

If $\Theta : \Gamma \Uparrow L$ has a proof of depth at most n in Σ_3 then $\Psi : \Phi \Uparrow \mathcal{G}$ is derivable in $\Sigma_3[\mathcal{P}]$.

Let's assume $\mathcal{R}(n)$ and let's show $\mathcal{R}(n+1)$. Let Π be a proof of depth $n+1$ of $\Theta: \Gamma \uparrow L$.

- (1) If L is not empty: say $L = L'$, $F_1 \wp F_2$ (the other cases for the last formula of L are similar). Therefore

$$\Pi = [\wp] \frac{\frac{\pi'}{\downarrow} \frac{\vdash \Theta: \Gamma \uparrow L', F_1, F_2}{\vdash \Theta: \Gamma \uparrow L', F_1 \wp F_2}}{\vdash \Theta: \Gamma \uparrow L', F_1 \wp F_2}$$

Let $\mathcal{G}' = \text{NGOAL}(L')$ and $g_1 = \text{NGOAL}(F_1)$ and $g_2 = \text{NGOAL}(F_2)$. By definition of the function NGOAL , we obtain that $\text{NGOAL}(F_1 \wp F_2) = g_1 \wp g_2$. Therefore, $\mathcal{G} = \mathcal{G}'$, $g_1 \wp g_2$. By the induction hypothesis applied to Π' we obtain that $\Psi: \Phi \uparrow \mathcal{G}'$, g_1, g_2 is derivable in $\Sigma_3[\mathcal{P}]$ and hence, so is $\Psi: \Phi \uparrow \mathcal{G}$ by application of $[\wp]$.

- (2) If L is empty, the last step of Π is necessarily an instance of a decision rule $[D_1]$ or $[D_2]$, say $[D_1]$

$$\Pi = [D_1] \frac{\frac{\pi'}{\downarrow} \frac{\vdash \Theta: \Gamma' \Downarrow F}{\vdash \Theta: \Gamma', F \uparrow}}{\vdash \Theta: \Gamma', F \uparrow}$$

where $\Gamma = \Gamma'$, F and F is not a positive atom. Let $u = \text{NATOM}(F)$. As F is not a positive atom, u is a Skolem constant and $\|\text{NMETH}(u^\perp \otimes F)\|$ is included in $\mathcal{P}$. Now, let's apply to the topmost layer of synchronous connectives of F the following transformations, which express associativity, commutativity and distributivity properties of the synchronous connectives.

$$\begin{array}{ll} (F_1 \otimes F_2) \otimes F_3 \mapsto F_1 \otimes (F_2 \otimes F_3) & (F_1 \oplus F_2) \oplus F_3 \mapsto F_1 \oplus (F_2 \oplus F_3) \\ F_1 \otimes F_2 \mapsto F_2 \otimes F_1 & F_2 \oplus F_2 \mapsto F_2 \oplus F_1 \\ 1 \otimes F \mapsto F & 0 \oplus F \mapsto F \\ F_1 \otimes (F_2 \oplus F_3) \mapsto (F_1 \otimes F_2) \oplus (F_1 \otimes F_3) & \\ 0 \otimes F \mapsto 0 & \end{array}$$

Notice that these transformations do not modify $\text{NMETH}(F)$ since the operators $*$ and $\cup$ on sets of quasi-clauses are associative commutative, have a neutral element (resp. $\{\{\emptyset; \emptyset\}\}$ and $\emptyset$) and $*$ is distributive over $\cup$. Furthermore, provability is also preserved, and even the focusing proofs, up to some irrelevant local reordering of the

inferences. Thus, for the distributivity property, we have

$$[\otimes] \frac{\frac{\vdots}{\vdash \Theta : \Gamma_1 \Downarrow F_1} [\oplus_l] \frac{\frac{\vdots}{\vdash \Theta : \Gamma_2 \Downarrow F_2} [\oplus_l] \frac{\vdots}{\vdash \Theta : \Gamma_2 \Downarrow F_2 \oplus F_3}}{\vdash \Theta : \Gamma_1, \Gamma_2 \Downarrow F_1 \otimes (F_2 \oplus F_3)}} \mapsto [\oplus_l] \frac{[\otimes] \frac{\frac{\vdots}{\vdash \Theta : \Gamma_1 \Downarrow F_1} \quad \frac{\vdots}{\vdash \Theta : \Gamma_2 \Downarrow F_2}}{\vdash \Theta : \Gamma_1, \Gamma_2 \Downarrow F_1 \otimes F_2}}}{\vdash \Theta : \Gamma_1, \Gamma_2 \Downarrow (F_1 \otimes F_2) \oplus (F_1 \otimes F_3)}$$

Hence, we may assume with no loss of generality that F is in normal form with respect to the transformation system above, i.e.

$$F = F_1 \oplus \dots \oplus F_n$$

where $n \geq 0$ (if $n = 0$ then $F = 0$) and each F_k is of the form

$$F_k = H_1 \otimes \dots \otimes H_m$$

where $m \geq 0$ (if $m = 0$ then $F_k = 1$) and the topmost connective of each H_j is neither of $0, 1, \oplus, \otimes$. By construction, Π' starts a critical focusing section, and is therefore necessarily of the form

$$\Pi' = [\oplus] \frac{\frac{\Pi'}{\vdash \Theta : \Gamma' \Downarrow F_k}}{\vdash \Theta : \Gamma' \Downarrow F}$$

where $k \in \{1 \dots n\}$. Let's make explicit among the component formulae H_j of F_k those which are negative atoms, say

$$F_k = p^\perp \otimes q^\perp \otimes H$$

where $p^\perp, q^\perp$ are negative atoms and H is a tensor product of formulae which either are positive atoms, or start with an asynchronous connective, or with the modality $!$ (since the other connectives are precluded). Let's take for instance

$$H = A \otimes !B$$

where A is a positive atom or an asynchronous formula and B is a formula. As the critical section proceeds, we have

$$\Pi'' = [\otimes] \frac{\frac{\frac{\frac{\Pi_p}{\vdash \Theta : \Gamma_p \Downarrow p^\perp} \quad \frac{\Pi_q}{\vdash \Theta : \Gamma_q \Downarrow q^\perp} \quad \frac{\Pi_A}{\vdash \Theta : \Gamma_A \Downarrow A} \quad \frac{\Pi_B}{\vdash \Theta : \Gamma_B \Downarrow !B}}{\vdash \Theta : \Gamma_p, \Gamma_q, \Gamma_A, \Gamma_B \Downarrow F_k}}}{\vdash \Theta : \Gamma_p, \Gamma_q, \Gamma_A, \Gamma_B \Downarrow F_k}}$$

where $\Gamma' = \Gamma_p, \Gamma_q, \Gamma_A, \Gamma_B$. Now, in Π_p and Π_q , a negative atom is reached which terminates the critical section. Hence, the identities must be used, say

$$\Pi_p = [I_2] \frac{}{\Theta : \Downarrow p^\perp} \quad \Pi_q = [I_1] \frac{}{\Theta : q \Downarrow q^\perp}$$

where $p \in \Theta$. Therefore $\Gamma_p = \emptyset$ and $\Gamma_q = q$. Furthermore, given that A is an asynchronous formula or a positive atom, we have

$$\Pi_A = [R\Downarrow] \frac{\frac{\vdots}{\vdash \Theta : \Gamma_A \Uparrow A}}{\vdash \Theta : \Gamma_A \Downarrow A} \quad \Pi_B = [!] \frac{\frac{\vdots}{\vdash \Theta : \Uparrow B}}{\vdash \Theta : \Downarrow !B}$$

Therefore $\Gamma_B = \emptyset$ and hence, $\Gamma = q, \Gamma_A, F$. Let $g_A = \text{NGOAL}(A)$ and $g_B = \text{NGOAL}(B)$ and $\Phi_A = \text{NATOM}(\Gamma_A)$. By the induction hypothesis we obtain that $\Psi : \Phi_A \Uparrow g_A$ and $\Psi : \Uparrow g_B$ are derivable in $\Sigma_3[\mathcal{P}]$. Now, by definition of the function NMETH , we obtain that

$$\text{NMETH}(F_k) = \{[p, q ; g_A, !g_B]\}$$

and hence, the method $M = u \wp p \wp q \multimap g_A \otimes !g_B$ is in $\|\text{NMETH}(u^\perp \otimes F)\|$, and hence in $\mathcal{P}$. Since $\text{NATOM}(p) = p$ and $p \in \Theta$, we obtain that $p \in \Psi$. Therefore, $\Psi : \Phi_A, u, q \Uparrow$ is derivable in $\Sigma_3[\mathcal{P}]$ by triggering method M :

$$[\circ-] \frac{[\otimes] \frac{[R\Downarrow] \frac{\frac{\vdots}{\vdash \Psi : \Phi_A \Uparrow g_A}}{\vdash \Psi : \Phi_A \Downarrow g_A} [!] \frac{\frac{\vdots}{\vdash \Psi : \Uparrow g_B}}{\vdash \Psi : \Downarrow !g_B}}{\vdash \Psi : \Phi_A \Downarrow g_A \otimes !g_B}}{\vdash \Psi : \Phi_A, u, q \Uparrow}$$

Now, since $\Gamma = \Gamma_A, q, F$, we obtain that $\Phi = \Phi_A, q, u$, and therefore $\Psi : \Phi \Uparrow$ is derivable in $\Sigma_3[\mathcal{P}]$.

Therefore $\mathcal{R}(n+1)$ holds. By induction $\mathcal{R}(n)$ holds for all n . $\square$

A.5. Cut elimination in the triadic system

A formula H is taken to be ‘reducible’ if any instance of the cut rule using H as cut formula can be eliminated from any proof. We have to show that all formulae are reducible. For simplification purpose, we only show here that formulae containing no modalities are reducible; the reasoning below could be adapted to avoid this restriction but it would become much more complicated.

DEMONSTRATION

To prove that any (modality-free) formula H is reducible, we reason by induction on H . Consider an occurrence of the cut rule

$$[\bowtie] \frac{\frac{\Pi}{\vdash \Theta : \Gamma \Downarrow H} \quad \frac{\Pi'}{\vdash \Theta : \Delta \Downarrow H^\perp}}{\vdash \Theta : \Gamma, \Delta \Uparrow}$$

where cuts have already been eliminated from Π and Π' . There are two cases in the induction.

- (1) If H is atomic, then, by symmetry, we may consider that H is a positive atom X . Therefore, Π is necessarily of the form

$$\Pi = [R\Downarrow, R\Uparrow] \frac{\vdots}{\frac{\vdash \Theta : \Gamma, X \Uparrow}{\vdash \Theta : \Gamma \Downarrow X}}$$

Hence, $\Theta : \Gamma, X \Uparrow$ is derivable. Moreover, as Π' is focusing, it is necessarily reduced to an identity. The two forms are possible:

- If

$$\Pi' = [I_1] \frac{}{\vdash \Theta : X \Downarrow X^\perp}$$

then $\Delta = X$ and $\Theta : \Gamma, \Delta \Uparrow$ is identical to $\Theta : \Gamma, X \Uparrow$ which is derivable.

- If

$$\Pi' = [I_2] \frac{}{\vdash \Theta : \Downarrow X^\perp}$$

then Δ is empty and X must be in Θ . But then, we can obtain a proof of $\Theta : \Gamma \Uparrow$ simply by discarding X from the proof of $\Theta : \Gamma, X \Uparrow$. The only case where this would not be possible is in a step where the identity rule $[I_1]$ is applied to match X and $X^\perp$, but such steps can be replaced by the identity rule $[I_2]$, which does not make use of X (since it is already in Θ). Thus, we obtain a proof of $\Theta : \Gamma \Uparrow$, which is identical to $\Theta : \Gamma, \Delta \Uparrow$ (since Δ is empty).

In both case, we obtain a (cut-free) proof of $\Theta : \Gamma, \Delta \Uparrow$. Therefore, atomic formulae are always reducible.

- (2) Let's now assume that H is not atomic and that all its strict sub-formulae are reducible. We want to show that H itself is reducible. By symmetry, we may assume that H is asynchronous. Let, for example, $H = A \wp (B \& C)$ be its topmost asynchronous layer (i.e. A, B, C are not asynchronous). The idea here is that the reduction of this topmost layer can be performed 'in one step'. Since Π and Π' are focusing, and A, B, C are not asynchronous, the two proofs are

necessarily of the form

$$\Pi = [\wp, \&] \frac{\frac{\vdots}{\vdash \Theta : \Gamma, A, B \uparrow} \quad \frac{\vdots}{\vdash \Theta : \Gamma, A, C \uparrow}}{\vdash \Theta : \Gamma \Downarrow A \wp (B \& C)}$$

and

$$\Pi' = [\otimes, \oplus_l] \frac{\frac{\vdots}{\vdash \Theta : \Delta_A \Downarrow A^\perp} \quad \frac{\vdots}{\vdash \Theta : \Delta_B \Downarrow B^\perp}}{\vdash \Theta : \Delta_A, \Delta_B \Downarrow A^\perp \otimes (B^\perp \oplus C^\perp)}$$

where $\Delta = \Delta_A, \Delta_B$ (notice that we have assumed here that the rule $[\oplus_l]$ is used to decompose $B^\perp \oplus C^\perp$; the other possibility $[\oplus_r]$ would be treated in the same way). Thus, $\Theta : \Delta_A \Downarrow A^\perp$ and $\Theta : \Delta_B \Downarrow B^\perp$ are derivable. Now, in the proof of $\Theta : \Gamma, A, B \uparrow$, let's replace the occurrences of A by Δ_A and those of B by Δ_B . This is always possible except in two cases (given here for A but B is treated in the same way):

- The Decision rule $[D_l]$ is used to select A :

$$[D_l] \frac{\frac{\vdots}{\vdash \Theta' : \Gamma' \Downarrow A}}{\vdash \Theta' : \Gamma', A \uparrow}$$

In this case, A can still be replaced by Δ_A , introducing a cut on A :

$$[\infty] \frac{\frac{\vdots}{\vdash \Theta' : \Gamma' \Downarrow A} \quad \frac{\vdots}{\vdash \Theta' : \Delta_A \Downarrow A^\perp}}{\vdash \Theta' : \Gamma', \Delta_A \uparrow}$$

- The identity rule $[I_l]$ is used to match A and $A^\perp$ (assuming A is a positive atom):

$$[I_l] \frac{}{\vdash \Theta' : A \Downarrow A^\perp}$$

In this case also, A can be replaced by Δ_A , yielding $\Theta' : \Delta_A \Downarrow A^\perp$ which is derivable.

Therefore, by replacing A by Δ_A and B by Δ_B , we obtain a proof of $\Theta : \Gamma, \Delta_A, \Delta_B \uparrow$ possibly containing cuts on A and B . But, as A and B are subformulae of H , by the induction hypothesis, they are reducible. Hence we can eliminate cuts on these formulae and obtain a cut-free proof of $\Theta : \Gamma, \Delta \uparrow$. Therefore, H is itself reducible. $\square$

